

Kubernetes Developer Learning Path

Version 1.0.0, 2026-07-20

Home

Introduction

Course Title: Kubernetes Developer Learning Path

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Overview
- Who is this for? pre-requisites.
- Week 0 — Familiarizing with koo·br·neh·teez
- Week 1.1 — Kubernetes Resource Model
- Week 1.2 — Controllers, informers, client-go
- Week 2.1 — Controllers contd.
- Week 2.2 — Going distributed: locks, leases, ownership, eventual consistency
- Week 3.1 — Taking more control: Admissions and beyond the kube-apiserver to writing your own API Server
- Week 3.2 — The other stuff
- Week 4.1 — All about kube-scheduler
- Week 4.2 — Ubiquitous topics!
- Exercises
- Recommended Reading
- Suggested Reading

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Overview

Overview

The Kubernetes Developer Learning Path provides a deep dive into the architecture and extensibility of Kubernetes, focusing on moving beyond basic usage to platform-level development. Key areas of focus include:

- **Control Plane Internals:** Understanding the API server lifecycle, authentication, authorization, and the role of the etcd persistence layer.
- **Platform Automation:** Mastering the creation of custom resources and operators using controller patterns, informers, and `client-go`.
- **Distributed System Design:** Implementing high-availability strategies, including leader election, locking, leasing, and idempotent state machines.
- **Advanced Architecture:** Exploring traffic management through the Gateway API, service meshes, and custom API server aggregation.
- **Operational Security & Scheduling:** Advanced cluster hardening, admission control webhooks, and the mechanics of the `kube-scheduler`.

Kubernetes architecture overview and extensibility mechanisms

Kubernetes Architecture Overview and Extensibility Mechanisms

Kubernetes is not just a container orchestrator; it is a highly extensible, API-driven distributed operating system. Understanding its architecture requires shifting your perspective from "command-execution" to "state-reconciliation."

The Control Plane: The Brain of the Cluster

At the heart of every Kubernetes cluster lies the **Control Plane**. Its primary role is to maintain the desired state of the cluster by processing API requests and orchestrating changes across worker nodes.

The Control Plane consists of four primary components:

- **kube-apiserver:** The front door to the cluster. Every action in Kubernetes—whether from `kubectl`, a controller, or an external tool—must pass through the API server.
- **etcd:** The consistent and highly-available key-value store used as Kubernetes' backing store for all cluster data.
- **kube-scheduler:** Watches for newly created Pods with no assigned node and selects a node for them to run on.
- **kube-controller-manager:** Runs controller processes that regulate the state of the cluster (e.g., node controllers, job controllers, and endpoint controllers).

The Declarative Model

Kubernetes operates on a **declarative configuration model**. Instead of telling the system **how** to perform a task (imperative), you tell the system **what** you want the state to be (declarative).

1. **Intent:** You submit a YAML/JSON manifest to the `kube-apiserver`.
2. **Storage:** The API server validates the object and persists it in `etcd`.
3. **Observation:** Controllers watch `etcd` for changes.
4. **Action:** Controllers perform actions to bridge the gap between the "Current State" and your "Desired State."

Extensibility Mechanisms: Making Kubernetes Your Own

The true power of Kubernetes lies in its extensibility. You are not limited to the built-in resources like `Pod` or `Service`. You can extend the API to handle custom logic through several mechanisms:

1. Custom Resource Definitions (CRDs)

CRDs allow you to define your own API objects. If you want to manage a "Database" or "MessageQueue" as a native Kubernetes object, you define a CRD. Once created, the `kube-apiserver` treats your custom resource as a first-class citizen, allowing you to use `kubectl` to manage them.

2. Controllers and Operators

A **Controller** is a control loop that watches the state of your cluster and makes changes to move the current state closer to the desired state. An **Operator** is a specialized controller that encodes operational knowledge (like how to back up or upgrade a database) into the controller's logic, automating complex day-two operations.

3. Admission Webhooks

These allow you to intercept requests to the `kube-apiserver` before they are persisted. * **Validating Webhooks:** Ensure a request meets your security or compliance standards (e.g., "All pods must have a resource limit"). * **Mutating Webhooks:** Modify the object before it is stored (e.g., "Automatically inject a sidecar container for logging").

4. API Aggregation

For advanced use cases, you can run your own API server alongside the `kube-apiserver`. The aggregation layer allows you to route requests to your custom server, enabling you to build fully independent APIs that leverage Kubernetes authentication and authorization.

Hands-on Activity: Observing the API Lifecycle

In this exercise, we will interact with the `kube-apiserver` to observe how declarative state is processed.

```
# 1. Create a simple declarative manifest
cat <<EOF > nginx-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-demo
spec:
  containers:
  - name: nginx
    image: nginx:latest
EOF

# 2. Submit the intent to the API server
kubectl apply -f nginx-pod.yaml

# 3. Observe the server's response lifecycle via logs (if you have cluster access)
# Note how the request passes through admission controllers before being persisted
kubectl get pod nginx-demo -o yaml
```

Challenge: Try deleting the `nginx-demo` pod and notice that you are not "killing the process" directly; you are submitting a request to the API server to **remove the intent** of that resource from `etcd`. The controller then observes this deletion and coordinates the cleanup across the nodes.

Control plane authentication, authorization, and validation

Control Plane Authentication, Authorization, and Validation

In a Kubernetes cluster, the `kube-apiserver` acts as the single entry point for all administrative, user, and controller-initiated requests. Ensuring that these requests are secure and valid is the primary function of the API server's request pipeline.

The API Request Lifecycle

When a request hits the `kube-apiserver`, it passes through three distinct phases:

1. **Authentication:** Identifying who is making the request.
2. **Authorization:** Determining what the identified user is allowed to do.
3. **Admission Control:** Validating or mutating the request content before it is persisted in `etcd`.

1. Authentication

The API server uses multiple authentication modules to verify identity. If a module succeeds, it returns a `User` object (including username, UID, and groups). If multiple modules are configured, the first to succeed wins.

- **X.509 Client Certs:** Used for inter-component communication (e.g., Kubelet to API server).
- **Static Token Files:** Simple, non-recommended for production.
- **OpenID Connect (OIDC):** Integration with identity providers (e.g., Keycloak, Dex, Okta).

- **Service Account Tokens:** JWT tokens assigned to Pods, used for cluster-internal communication.

2. Authorization

Once authenticated, the system checks if the user has permission to perform the requested action (Verb: `get`, `list`, `create`, `delete`, etc.) on a specific Resource/Subresource.

- **RBAC (Role-Based Access Control):** The standard mechanism using `Roles/ClusterRoles` and `RoleBindings/ClusterRoleBindings`.
- **ABAC (Attribute-Based Access Control):** Policy-based, configured via local files.
- **Webhook Authorization:** The API server delegates the decision to an external service via a POST request.

3. Admission Control

Admission controllers are code modules that intercept requests **after** they are authenticated and authorized, but **before** the object is persisted.

- **Mutating Admission Controllers:** Can modify the content of the request (e.g., injecting sidecars or default labels).
- **Validating Admission Controllers:** Can reject a request if it violates cluster policies (e.g., "all images must come from our internal registry").

Hands-on Activity: Configuring RBAC and Validation

In this exercise, you will create a restricted ServiceAccount and attempt to enforce a simple check.

Step 1: Create a restricted ServiceAccount

Create a ServiceAccount that has limited permissions to list pods in a specific namespace.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pod-lister
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: RoleBinding
metadata:
  name: read-pods
subjects:
- kind: ServiceAccount
  name: pod-lister
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Step 2: Test Authorization

Use the `auth can-i` command to verify the permissions of your new ServiceAccount:

```
kubectl auth can-i list pods --as=system:serviceaccount:default:pod-lister
# Expected output: yes
kubectl auth can-i delete pods --as=system:serviceaccount:default:pod-lister
# Expected output: no
```

Step 3: Discussion on Validation

In a production scenario, RBAC is not enough. If you wanted to prevent users from creating Pods that use the `latest` image tag, you would implement a **Validating Admission Webhook**.

- **Why use a Webhook instead of RBAC?** RBAC controls **who** can do something; Admission Webhooks control **what** is valid within the object schema.



For further exploration, refer to the **Kubernetes Security and Observability** guide provided in the recommended reading list to understand how to audit these control plane requests effectively.

Platform automation through custom resources and operators

Platform Automation: Custom Resources and Operators

In the Kubernetes ecosystem, "Platform Automation" refers to the ability to extend the platform's native capabilities to manage domain-specific applications and infrastructure. Instead of manually orchestrating complex systems, we encode operational knowledge into software using Custom Resources (CRDs) and Controllers (Operators).

The Kubernetes Extension Pattern

Kubernetes is built on the philosophy that everything is an API object. When the standard set of resources (Pods, Deployments, Services) is insufficient for your business logic, you can extend the API.

- **Custom Resource Definition (CRD):** Defines the "schema" or the **Desired State** for your custom objects.
- **Controller/Operator:** The "brain" that constantly watches the state of the cluster and performs actions to align the **Actual State** with the **Desired State**.

An **Operator** is a specialized controller that automates the lifecycle of complex, stateful applications (e.g., databases, message queues) by embedding human operational knowledge—such as backups, upgrades, and scaling policies—directly into the reconciliation loop.

Reconciliation: The Heart of Automation

The core of every operator is the **Reconciliation Loop**. Unlike imperative scripts that run once and finish, a controller follows a declarative, asynchronous pattern:

1. **Observe:** Get the state of the resource from the `kube-apiserver` (via caches/informers).
2. **Analyze:** Compare the current cluster state against the `spec` defined in the custom resource.
3. **Act:** Perform the necessary operations (CRUD) to reach the desired state.
4. **Repeat:** Wait for the next event or requeue based on a change.

Hands-On Activity: Scaffolding an Operator

In this activity, we will use `kubebuilder` to scaffold the foundation of an operator.

Prerequisites

- `go` (v1.20+)
- `docker` or `podman`
- `kubectl`
- `kubebuilder` installed

Step 1: Initialize the Project

Create a new directory and initialize your project:

```
mkdir my-operator
cd my-operator
kubebuilder init --domain example.com --repo github.com/example/my-operator
```

Step 2: Create a Custom API

Define a new resource named `DatabaseBackup` to automate backup tasks:

```
kubebuilder create api --group storage --version v1alpha1 --kind DatabaseBackup
```

This command generates: `* api/v1alpha1/databasebackup_types.go`: Where you define the schema (`Spec` and `Status`). `* internal/controller/databasebackup_controller.go`: Where the reconciliation

logic resides.

Step 3: Implement the Reconcile Logic

Open `internal/controller/databasebackup_controller.go` and locate the `Reconcile` function. This is where you write your automation logic:

```
func (r *DatabaseBackupReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    // 1. Fetch the DatabaseBackup instance
    // 2. Add logic to trigger backup (e.g., executing a command or API call)
    // 3. Update Status field with success/failure
    return ctrl.Result{}, nil
}
```

Step 4: Deploy and Test

1. **Install CRDs:** `make install`
2. **Run the controller locally:** `make run`
3. **Apply a sample resource:** Create `config/samples/storage_v1alpha1_databasebackup.yaml` and run `kubectl apply -f config/samples/...`

Best Practices for Operators

- **Idempotency:** Always ensure your reconcile function can run multiple times without causing side effects.
- **Spec vs Status:** Never let the controller update the `Spec`. The user owns the `Spec`; the controller owns the `Status`.
- **Events:** Use `record.EventRecorder` to emit Kubernetes events so users can see what the operator is doing via `kubectl describe`.
- **Lease/Locking:** When running multiple replicas of an operator, use `leader election` (via `client-go` Leases) to ensure only one instance reconciles a specific resource at a time.

Further Learning

For deep dives into operator architecture, refer to: * **Kubernetes Operators** by Dobies & Wood. * The [CNCF Operator Whitepaper](<https://tag-app-delivery.cncf.io/whitepapers/operator/>).

Distributed system principles for high availability

Distributed System Principles for High Availability in Kubernetes

In a Kubernetes environment, high availability (HA) is not merely a feature; it is an architectural requirement. As we scale controllers and custom operators, we move from simple automation to distributed systems engineering. This section covers the fundamental principles of building resilient, concurrent control loops.

The Challenge of Distributed State

When multiple controller instances run across a cluster, they must operate on shared state without causing corruption or "split-brain" scenarios. Kubernetes relies on **eventual consistency**, where the system continuously drives the actual state toward the desired state defined in the API.

To achieve this in distributed controllers, we must master:

- * **Idempotency**: The ability of an operation to be applied multiple times without changing the result beyond the initial application.
- * **Optimistic Concurrency**: Using `resourceVersion` to prevent "lost updates" where one controller overwrites changes made by another.
- * **Leader Election**: Ensuring that for a specific set of resources, only one controller instance is actively performing the reconciliation loop.

Mechanisms for Coordination

Locks and Leases

Kubernetes provides the `Lease` API (under `coordination.k8s.io`) to manage distributed locks. A lease acts as a heartbeat mechanism: if a controller holds the lease, it is the designated leader. If the heartbeat stops (e.g., the pod crashes), the lease expires, and another instance can acquire it.

Controller Sharding

For high-scale environments (like the OpenShift Ingress Controller), running a single controller instance may become a bottleneck. Sharding allows you to partition the workload.

- * **Consistent Hashing**: Distributing work based on resource names or IDs.
- * **Workqueues with Partitioning**: Controllers only process items they "own" based on specific labels or annotations.

Hands-on Activity: Implementing Leader Election

This exercise demonstrates how to ensure only one instance of your controller performs critical tasks.



Ensure your ServiceAccount has `get`, `create`, and `update` permissions on the `leases` resource in the `coordination.k8s.io` group.

1. Step 1: Initialize the LeaderElection Config Using `client-go`, create a `LeaderElectionConfig`:

```
import "k8s.io/client-go/tools/leaderelection"

config := leaderelection.LeaderElectionConfig{
    Lock:          lock, // An instance of resourceLock.LeaseLock
    ReleaseOnCancel: true,
    LeaseDuration:  15 * time.Second,
    RenewDeadline:  10 * time.Second,
    RetryPeriod:    2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // Run your controller logic here
            runController(ctx)
        },
    },
}
```

```
OnStoppedLeading: func() {  
    log.Fatalf("Lost leadership")  
},  
},  
}
```

1. **Step 2: Start the Election** Invoke the `RunOrDie` method to block until the leader election is complete:

```
leaderelection.RunOrDie(ctx, config)
```

1. **Step 3: Verification** Deploy two replicas of your controller. Observe the `Lease` object in the namespace:

```
kubectl get lease -n <your-namespace>  
kubectl describe lease <lease-name>
```

You will see the `HolderIdentity` field reflecting which pod currently holds the lock.

Best Practices for Distributed Controllers

- **Avoid "Thundering Herd":** Use jitter in your requeue intervals to prevent all controller instances from slamming the `kube-apiserver` at the exact same millisecond.
- **Always use Informer Caches:** Never query the API server directly in your reconcile loop; always use the local cache provided by `client-go` Informers to reduce load.
- **Fail Fast:** If your controller cannot satisfy its preconditions, requeue the object and move to the next, rather than blocking the entire queue.



For deeper architectural patterns, consult **Designing Distributed Systems** by Brendan Burns. It provides the foundational logic for the sidecar and controller patterns we utilize in Kubernetes today.

Advanced networking, traffic management, and control plane architectures

Advanced Networking, Traffic Management, and Control Plane Architectures

Overview

As Kubernetes clusters grow beyond simple deployments, the complexity of managing traffic and control plane topology increases. This module explores how to move beyond basic `ClusterIP` services toward sophisticated traffic routing and how to adapt the Kubernetes control plane for varied environments, ranging from massive multi-tenant systems to resource-constrained edge

devices.

Advanced Networking and Traffic Management

Modern Kubernetes networking relies on abstracting complex service communication. While the legacy **Ingress** resource served as an early standard, the industry has shifted toward the **Gateway API**, which provides a more expressive, role-oriented set of primitives.

Service Mesh Concepts

A service mesh (e.g., Istio, Linkerd) introduces a "sidecar" or "sidecar-less" proxy layer to handle the challenges of microservices communication, including:

- * **mTLS (mutual TLS)**: Automating identity and encryption for all service-to-service traffic.
- * **Traffic Splitting**: Canary deployments and blue-green releases by shifting percentages of traffic at the L7 (Application) layer.
- * **Observability**: Capturing golden signals (latency, errors, traffic, saturation) without modifying application code.

The Gateway API

Unlike the **Ingress** resource, the Gateway API defines a hierarchy of resources:

- * **GatewayClass**: Defines the type of load balancer or proxy implementation.
- * **Gateway**: A logical instance of a load balancer listener.
- * **HTTPRoute**: Defines how traffic is routed to services based on headers, paths, or hostnames.

Alternative Control Plane Architectures

Standard Kubernetes architectures are not one-size-fits-all. Depending on the scale, multi-tenancy requirements, or hardware constraints, engineers often employ specialized control plane form factors.

1. Hosted and Nested Architectures (HyperShift)

HyperShift decouples the control plane from the worker nodes. By running the control plane as a workload (pods) on a management cluster, you reduce the operational overhead of managing master nodes for every single tenant cluster. This is often referred to as "Hosted" or "Nested" OpenShift.

2. Multi-tenant and Meta-level Control Planes (kcp)

kcp acts as a Kubernetes-like control plane that supports multiple "workspaces" (virtual clusters). It provides a way to manage resources across many physical clusters from a single API server, acting as a "control plane for control planes." It is ideal for large-scale multi-tenancy where users need an isolated view of resources without the overhead of spinning up entire cluster control planes.

3. Edge and Lightweight Architectures

- **MicroShift**: An enterprise-grade, optimized version of OpenShift designed specifically for small-footprint edge devices. It strips away high-overhead components while maintaining the API

compatibility required for consistency with larger clusters.

- **k3s / k0s / microk8s:** These distributions optimize the control plane by packaging components into a single process, reducing memory and CPU footprints for IoT or CI/CD runner environments.

Hands-on Activity: Exploring Gateway API Primitives

In this activity, we will examine how to route traffic using the Gateway API model.



Ensure you have a cluster with a Gateway API-compatible controller installed (e.g., Istio, Cilium, or Contour).

1. **Step 1: Define a Gateway** Create a **Gateway** resource that defines where your traffic enters the cluster.

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: edge-gateway
spec:
  gatewayClassName: istio
  listeners:
  - name: http
    protocol: HTTP
    port: 80
```

1. **Step 2: Define an HTTPRoute** Route traffic for **api.example.com** to your backend service.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api-route
spec:
  parentRefs:
  - name: edge-gateway
  hostnames:
  - "api.example.com"
  rules:
  - backendRefs:
    - name: my-service
      port: 8080
```

1. **Step 3: Verification** Apply these manifests and use **kubectl get gateway** to verify the status. Check that the **ADDRESS** field is populated, indicating the load balancer has successfully attached to your configuration.

Troubleshooting Tips

- **Traffic Flow:** If traffic is failing, verify the `status` block of the `HTTPRoute`. It will often provide `ParentRef` conditions explaining if the route was successfully attached to the Gateway.
- **Control Plane Mismatch:** When dealing with lightweight control planes like MicroShift, always ensure your local `kubectl` context is pointing to the correct API endpoint, as these distributions often run on non-standard ports (e.g., 6443 vs 8443).

Who is this for? pre-requisites.

Who is this for? Pre-requisites

This learning path is designed for engineers, system administrators, and developers who already possess practical experience working with containerized environments.

To ensure success, participants should have:

- **Recommended background:** Basic knowledge of distributed systems principles.
- **Baseline competency:** Proficiency equivalent to the Certified Kubernetes Application Developer (CKAD) certification, or completion of Red Hat DO-180 and DO-280 training tracks.

Target audience: Engineers, sys admins, and developers with container experience

Prerequisites and Target Audience

Before embarking on this learning path, it is essential to ensure that you have the foundational knowledge required to digest the architectural complexities of the Kubernetes control plane. This curriculum is designed for technical practitioners who are ready to move beyond basic container management and transition into platform engineering and cluster-level development.

Who is this for?

This learning path is specifically tailored for:

- **Platform Engineers:** Professionals tasked with building internal developer platforms, managing multi-cluster environments, and extending Kubernetes functionality.
- **Software Engineers:** Developers who need to understand how to interact programmatically with the Kubernetes API, develop custom operators, or build controller-based automation.
- **System Administrators:** Sysadmins looking to deepen their expertise in cluster internals, security hardening, and advanced networking patterns to support robust production environments.

Recommended Prerequisites

While this path is comprehensive, it assumes you are not a beginner to the ecosystem. To succeed, you should be comfortable with:

- **Container Fundamentals:** A solid understanding of container runtimes (e.g., CRI-O, containerd), images, and basic container lifecycles.
- **Kubernetes Basics:** Competency in deploying and managing standard workloads (Pods, Deployments, Services, ConfigMaps).
- **Distributed Systems Knowledge:** An awareness of fundamental concepts such as consistency,

consensus, replication, and network partitions.

- **Go Programming (Recommended):** Since much of the Kubernetes ecosystem—including `client-go` and `controller-runtime`—is written in Go, familiarity with the language will significantly lower the barrier to entry for the hands-on exercises.

Ideal Baseline

To gauge your readiness, we recommend the following as an ideal entry point:

1. **Certification level:** Proficiency equivalent to the **Certified Kubernetes Application Developer (CKAD)**.
2. **Red Hat Training Tracks:** Successful completion of the **DO-180 (Introduction to Containers, Kubernetes, and Red Hat OpenShift)** and **DO-280 (Red Hat OpenShift Administration: Core)** tracks provides the most relevant background for the advanced topics covered in this course.

Self-Assessment Checklist

If you can answer "Yes" to the following questions, you are prepared to begin Week 0:

- ☐ Can I explain the difference between imperative and declarative state management?
- ☐ Do I understand the basic request flow from `kubectl` to the Kubelet?
- ☐ Am I comfortable reading YAML-based resource manifests?
- ☐ Do I have a development environment ready, including a local Kubernetes cluster (e.g., Kind or Minikube)?



If you feel your knowledge in these areas is thin, we strongly suggest reviewing the official Kubernetes documentation on "Concepts" before proceeding to the architecture and extensibility modules.

Recommended prerequisites: Basic distributed systems knowledge

Recommended Prerequisites: Mastering Distributed Foundations

To succeed in this Kubernetes Developer Learning Path, you must bridge the gap between container orchestration and the underlying principles of distributed systems. Because Kubernetes is, at its core, a distributed system platform, your ability to reason about the cluster's behavior relies on your understanding of how independent, networked components maintain state and consistency.

Understanding the Baseline

While basic container knowledge (Docker/Podman) is a foundational requirement, this course moves beyond the lifecycle of a single pod. You are expected to have:

- **Experience with Container Orchestration:** Familiarity with concepts like Pods, Deployments, and Services. If you hold a **CKAD certification** or have completed the **Red Hat DO-180 and DO-**

280 tracks, you possess the ideal baseline.

- **Systems Programming Awareness:** Experience with at least one language commonly used in Kubernetes development, preferably **Go (Golang)**, as it is the language of the Kubernetes control plane and `client-go`.

Why Distributed Systems Knowledge Matters

Kubernetes operates on an **eventual consistency** model rather than a strict transactional one. Before diving into `client-go` informers or writing custom controllers, you should be comfortable with the following core distributed concepts:

1. State Reconciliation vs. Imperative Execution

Unlike a script that executes commands sequentially, Kubernetes controllers operate on a control loop (reconcile loop). You must understand the transition from **imperative** actions ("do this now") to **declarative** state management ("make sure the world looks like this").

2. The CAP Theorem and Consistency

In a distributed environment, you are balancing Consistency, Availability, and Partition Tolerance. Kubernetes prioritizes consistency at the storage layer (`etcd`) while providing high availability through replication. You will be interacting with systems where: * **Network partitions occur:** How does your controller react when the API server is unreachable? * **Optimistic Concurrency:** Kubernetes uses resource versioning (`metadata.resourceVersion`) to handle concurrent updates rather than traditional database locking.

3. Idempotency

In a system where controllers may crash and restart, or where network events trigger multiple reconciliations, every action your code takes must be **idempotent**. * **Definition:** An operation is idempotent if it can be performed multiple times without changing the result beyond the initial application. * **Relevance:** You will learn to design state machines that calculate the "delta" between the current state and the desired state, rather than blindly executing commands.

4. Handling Concurrency and Synchronization

When managing cluster-wide state, you must ensure that only one "source of truth" acts on a resource at a time. This involves understanding: * **Leader Election:** Utilizing the `Lease` API to ensure only one controller instance is active in a high-availability deployment. * **Distributed Locking:** Using Kubernetes primitives to prevent race conditions when multiple controllers or processes compete for the same resource.

Recommended Pre-study Resources

To prepare for the advanced architectural topics in this path, we recommend reviewing the following conceptual frameworks:

- **Brendan Burns, Designing Distributed Systems:** Focus on the "Sidecar," "Ambassador," and "Work Queue" patterns. These patterns form the structural backbone of how we design Kubernetes-native applications.

- **The Go Concurrency Model:** Review goroutines, channels, and the `context` package. `client-go` relies heavily on these to manage the lifecycle of informers and asynchronous event handling.



If you feel your distributed systems foundation is shaky, focus on the "Reconciliation Loop" pattern first. This is the single most important mental model for a Kubernetes Developer. Everything else—from `client-go` informers to Custom Resource Definitions (CRDs)—is built to support or leverage this loop.

Ideal baseline: CKAD certification, or Red Hat DO-180 and DO-280 tracks

Recommended Prerequisites: Building Your Foundation

To successfully navigate this Kubernetes Developer Learning Path, it is essential to establish a solid foundation in container orchestration and Kubernetes primitives. This program assumes you are moving beyond basic usage and are preparing to engineer custom extensions, controllers, and complex system architectures.

The Ideal Baseline

We recommend that participants possess a baseline of knowledge equivalent to one of the following industry-recognized certification tracks. These paths ensure you are comfortable with the core API objects (Pods, Deployments, Services) and the imperative vs. declarative workflows before we dive into the internal mechanics of the control plane.

Option 1: CKAD (Certified Kubernetes Application Developer)

The **CKAD** certification confirms your ability to design, build, and deploy cloud-native applications for Kubernetes. * **Why this helps:** The exam focuses on application lifecycle management, configuration, multi-container pods, and observability. * **Key competencies acquired:** * Proficiency with `kubectl` and manifest management. * Understanding of ConfigMaps, Secrets, and persistent volumes. * Practical experience in application troubleshooting within a cluster environment.

Option 2: Red Hat Training Tracks (DO-180 & DO-280)

If you prefer a structured, vendor-aligned approach, the Red Hat curriculum provides a deep dive into OpenShift (Kubernetes) internals.

- **DO-180: Introduction to Containers, Kubernetes, and Red Hat OpenShift**
- This is the entry point for understanding the container runtime, image creation, and basic pod deployment. It provides the "how-it-works" for containerization that is prerequisite to understanding how Kubernetes manages those containers at scale.
- **DO-280: Red Hat OpenShift Administration II: Operating a Production Kubernetes Cluster**
- This track transitions from application development to platform operations. It covers crucial topics such as:

- Advanced routing and ingress management.
- Resource quotas and limits.
- Cluster security, including Role-Based Access Control (RBAC).
- Troubleshooting the platform rather than just the application.

Skillset Gap Analysis

Before beginning this course, ensure you can comfortably answer "Yes" to the following:

Competency	Requirement
Containerization	Can you build, tag, and push a container image (OCI-compliant) to a registry?
Manifests	Are you comfortable writing YAML manifests for standard K8s resources from scratch?
Observability	Can you interpret application logs and <code>kubectl describe</code> outputs to debug a crash-looping pod?
Networking	Do you understand the relationship between a Service (ClusterIP/NodePort) and its underlying Pod endpoints?



If you feel your knowledge is limited to "running" applications but you are unfamiliar with the **lifecycle** of a resource, we highly recommend reviewing the **Kubernetes.io** documentation on "API Concepts" and "Workloads" before starting the Week 0 modules.

Week 0 — Familiarizing with koo·br·neh·teez

Week 0 — Familiarizing with Kubernetes

This foundational phase introduces the core architectural principles that govern Kubernetes. Participants learn how the system leverages an API-centric, declarative configuration model to manage infrastructure state.

Key concepts include:

- **Kubernetes Control Plane:** Understanding the role of the API server as the central hub for cluster communication and coordination.
- **Persistence Layer:** Exploring `etcd` as the primary key-value store for maintaining cluster state.
- **Asynchronous Reconciliation:** An introduction to the control loop pattern, where the system continuously observes and adjusts the current state to match the declared desired state.

API-centric declarative configuration management

API-Centric Declarative Configuration Management

Kubernetes is fundamentally an API-centric system. Unlike traditional infrastructure management where you might run a sequence of imperative commands (e.g., "start container X," "open port Y"), Kubernetes operates by observing the **desired state** provided by the user and reconciling it against the **actual state** of the cluster.

The Declarative Paradigm

In a declarative system, you describe the **what**, not the **how**. You submit a manifest—usually in YAML or JSON—to the Kubernetes API server, stating exactly what your application should look like.

- **Imperative:** "Run this container, then scale it to three replicas, then attach a storage volume."
- **Declarative:** "I want a `Deployment` object with three replicas, using image `nginx:latest`, and a `PersistentVolumeClaim` attached."

The Kubernetes control plane treats this manifest as the "source of truth." Once submitted, the system continuously works to ensure that the cluster's current state matches your declared intent. If a container crashes, the controller notices the discrepancy between the desired state (3 replicas) and the actual state (2 replicas) and automatically spins up a new instance.

The Heart of the System: The API Server

The `kube-apiserver` acts as the central gateway for all interactions. It provides a RESTful interface that allows components to communicate, store, and retrieve state.



The API server is the only component that communicates directly with `etcd`, the persistent key-value store. By acting as the sole interface for state changes, the API

server ensures that all cluster operations go through validation, authentication, and authorization before being persisted.

The Request Lifecycle

When you send a configuration to the cluster (e.g., via `kubectl apply`), the following high-level process occurs:

1. **Authentication/Authorization:** The `kube-apiserver` verifies who you are and if you have permissions to perform the action.
2. **Admission Control:** Webhooks may inspect, reject, or modify the request (more on this in Week 3.1).
3. **Persistence:** Upon successful validation, the object is stored in `etcd`.
4. **Reconciliation:** Controllers (watching the API server) observe the new state and perform the necessary work (like scheduling pods or mounting volumes) to bring the cluster to that desired state.

Hands-On Activity: Observing Declarative State

In this exercise, you will observe how the API server manages declarative intent.

Prerequisites

- A running Kubernetes cluster (e.g., Minikube, Kind, or OpenShift).
- `kubectl` installed and configured.

Step 1: Create a Declarative Manifest

Create a file named `nginx-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
```

```
image: nginx:1.14.2
ports:
- containerPort: 80
```

Step 2: Apply the Intent

Apply the configuration to your cluster:

```
kubectl apply -f nginx-deployment.yaml
```

Step 3: Observe the Reconciliation

After applying, verify that the system is acting on your behalf:

```
kubectl get deployments
kubectl get pods
```

Step 4: Simulate a Failure

Manually delete one of the pods managed by your deployment:

```
# Replace <pod-name> with the actual name of one of your nginx pods
kubectl delete pod <pod-name>
```

Observe the system behavior immediately after:

```
kubectl get pods
```

Expected Result: You will see that Kubernetes has already initiated a new pod to replace the one you deleted. This confirms the **declarative** nature of the system: the `kube-apiserver` and its controllers are constantly maintaining the state you declared in your YAML file, regardless of external disruptions.

Kubernetes control plane and API server role

Kubernetes Control Plane and API Server Role

The Kubernetes control plane is the "brain" of the cluster. It orchestrates the cluster state, manages scheduling, and maintains the desired state of all workloads. At the heart of this system lies the `kube-apiserver`, the front door to the cluster that exposes the Kubernetes API.

The Control Plane Architecture

The control plane consists of several core components working in concert:

- **kube-apiserver:** The central hub that processes REST requests, performs authentication/authorization, and persists cluster state to **etcd**.
- **etcd:** A strongly consistent, distributed key-value store used as Kubernetes' backing store for all cluster data.
- **kube-scheduler:** Watches for newly created Pods with no assigned node and selects a node for them to run on.
- **kube-controller-manager:** Runs controller processes (e.g., Node Controller, Job Controller) that regulate the state of the cluster by reconciling current state with desired state.

The Role of kube-apiserver

The **kube-apiserver** is the only component that communicates directly with **etcd**. All other components (and external users) communicate exclusively with the API server.

The Request Lifecycle

When a request enters the **kube-apiserver**, it traverses a series of plugin layers before reaching storage. This pipeline ensures security, integrity, and operational fairness:

1. **Authentication:** Validates the identity of the user or service account (e.g., OIDC, X.509 certs, or tokens).
2. **Audit:** Logs the request details for security and compliance.
3. **Impersonation:** Allows users to perform actions on behalf of another user if authorized.
4. **Priority & Fairness:** Manages request flow to prevent the API server from being overwhelmed.
5. **Authorization:** Determines if the authenticated user has permission to perform the specific action (RBAC, ABAC, Webhooks).
6. **Admission:** Enforces policies via **Validating** or **Mutating** webhooks.
7. **Storage:** Writes the object to **etcd** or retrieves the requested data.



Admission webhooks (Validating and Mutating) are executed only during **CREATE**, **UPDATE**, or **DELETE** operations. They are **not** triggered during **GET** or **LIST** operations.

Hands-on Activity: Inspecting the API Flow

In this exercise, you will observe the API server's role in processing a simple resource creation.

Prerequisites

- A running Kubernetes cluster (e.g., Minikube, Kind, or OpenShift).
- **kubectl** installed and configured.

Step 1: Enable Audit Logging

To see how the API server processes requests, you can look at the audit logs. If you have access to the control plane, inspect the log file (usually defined in the `/etc/kubernetes/manifests/kube-apiserver.yaml` configuration).

```
# Check the audit log location
grep "audit-log-path" /etc/kubernetes/manifests/kube-apiserver.yaml
```

Step 2: Create a Resource and Trace

Create a simple ConfigMap and observe how the API server registers the intent.

1. Create the manifest:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: test-config
data:
  key: value
```

1. Apply it with verbose output to see the interaction:

```
kubectl apply -f configmap.yaml -v=9
```

1. Observe the **PUT** or **POST** requests being sent to the `/api/v1/namespaces/default/configmaps/test-config` endpoint.

Step 3: Verification

Use `kubectl` to confirm that the `kube-apiserver` has persisted the object to `etcd`:

```
# Check the status of the resource
kubectl get configmap test-config -o json
```



For deep dives into the lifecycle, refer to the "Life (or Death) of a Kubernetes API Request" documentation, which maps every step of the plugin layer chain.

Persistence layer: etcd key-value store

Persistence Layer: The etcd Key-Value Store

In the Kubernetes architecture, the `kube-apiserver` acts as the gateway to the cluster, but the "source of truth" resides in `etcd`. Understanding how Kubernetes utilizes `etcd` is fundamental to grasping

the distributed nature of the system.

Overview of etcd

etcd is a distributed, consistent, key-value store used to store all cluster data. It is not just a database for Kubernetes; it is the heartbeat of the control plane. When you interact with the Kubernetes API, you are modifying the state stored within **etcd**.

Key characteristics include:

- * **Consistency:** Based on the Raft consensus algorithm, ensuring that even in a distributed setup, all nodes agree on the state of the data.
- * **Key-Value Structure:** Kubernetes stores resources as serialized JSON or Protobuf objects, keyed by their path (e.g., `/registry/pods/default/my-pod`).
- * **Watch Mechanism:** **etcd** provides an efficient watch API that allows the **kube-apiserver** (and consequently, informers in controllers) to receive notifications when keys change, enabling the reactive, asynchronous nature of Kubernetes controllers.

How Kubernetes Uses etcd

When a user submits a manifest (a declarative configuration), the **kube-apiserver** performs the following steps:

1. **Validation:** Checks the request against schemas and webhooks.
2. **Conversion:** Converts the internal representation of the resource.
3. **Storage:** Writes the object into **etcd**.

Important Architectural Considerations

- **Vertical Scaling:** It is critical to note that for the **etcd** and **kube-apiserver** control plane components, **vertical scaling** (adding more CPU/RAM to the existing nodes) is generally preferred over horizontal scaling. Expanding the **etcd** cluster size beyond 3 or 5 members often introduces latency due to the consensus overhead.
- **resourceVersion:** Every object in **etcd** has a `metadata.resourceVersion`. This acts as a versioning mechanism for optimistic concurrency control. When a controller attempts an update, it must ensure its version matches the current version in **etcd**, preventing "lost updates" in a multi-writer environment.

Hands-on Activity: Observing etcd Keys

This activity requires an existing Kubernetes cluster where you have access to the **etcd** data directory or the **etcdctl** utility.

Prerequisites

- Access to a Kubernetes control plane node.
- **etcdctl** installed.
- Environment variables set for **etcd** certs:
- `ETCDCTL_API=3`
- `ETCDCTL_CACERT`
- `ETCDCTL_CERT`
- `ETCDCTL_KEY`

Step 1: Querying the etcd store

To see how objects are stored, we will list the keys under the Kubernetes registry path.

```
# List all resources stored in the cluster registry
etcdctl get /registry --prefix --keys-only
```

Step 2: Inspecting a specific resource

Pick a pod name from your cluster and inspect its serialized representation.

```
# Replace <namespace> and <pod-name> with your values
etcdctl get /registry/pods/<namespace>/<pod-name> --print-value-only
```

Step 3: Observing the Change

1. Open a new terminal and "watch" the registry:

```
etcdctl watch /registry/pods/<namespace>/<pod-name>
```

1. In your primary terminal, update a label on that pod via `kubectl`.
2. Observe the output in your `etcdctl watch` window. You will see the serialized JSON object being updated in real-time.

Troubleshooting Tips

- **Slow etcd Performance:** Often caused by high disk I/O latency. Ensure `etcd` is backed by high-performance SSDs.
- **Compaction:** `etcd` keeps a history of revisions. If the history grows too large, it consumes excessive memory. Kubernetes automatically performs compaction, but monitoring `etcd_db_total_size_in_bytes` is vital for cluster health.
- **Watch Cache:** The `kube-apiserver` maintains a memory-resident "watch cache" to satisfy requests without hitting `etcd` directly for every read. If you see high latency on reads, verify the `kube-apiserver` cache metrics.

Asynchronous controller reconciliation patterns

Asynchronous Controller Reconciliation Patterns

The power of Kubernetes lies in its ability to transition a system from its current state to a desired state without human intervention. This capability is powered by the **Reconciliation Pattern**, an asynchronous control loop that continuously monitors the state of the cluster and acts to resolve discrepancies.

The Control Loop Mechanism

At the heart of every Kubernetes controller is an asynchronous control loop. Unlike traditional procedural scripts that execute tasks in a linear sequence, a controller acts as an autonomous agent that follows a cycle of **Observe, Analyze, and Act**.

```
for {  
  1. Observe: Get current state from the API Server (via Cache/Informer).  
  2. Analyze: Compare current state with the desired state (Spec).  
  3. Act: Execute operations (CRUD) to align current state with desired state.  
}
```

Key Characteristics

- **Asynchronous:** The controller does not wait for a specific request to trigger an action; it responds to changes in the cluster state whenever they occur.
- **Declarative:** The controller focuses on **what** the end state should be (the Spec), not **how** to get there.
- **Idempotency:** Because the loop runs repeatedly, it is critical that actions are idempotent. Applying the same action multiple times must result in the same state, preventing side effects or resource duplication.

Event Propagation and Synchronization

In a large cluster, the API server would be overwhelmed if every controller requested the state of all resources every few milliseconds. To solve this, Kubernetes uses a **Shared Informer** architecture.

The Role of the Cache

Controllers do not talk directly to the `kube-apiserver` for every reconciliation cycle. Instead, they utilize a local cache maintained by `client-go`. 1. **List and Watch:** The informer performs an initial `List` of objects and then opens a `Watch` stream to receive incremental updates (Add, Update, Delete events). 2. **Event Handling:** These events are processed and stored in a local, thread-safe cache (store). 3. **Efficiency:** When the controller needs the current state, it fetches the object from the local cache rather than making a network call to the API server.

Hands-on Activity: Observing Reconciliation

In this activity, we will simulate the conceptual flow of a reconciliation loop.

Prerequisites

- A running Kubernetes cluster (e.g., Minikube or Kind).
- `kubectl` installed and configured.

Step 1: Simulate the "Desired State"

Create a simple manifest that represents our desired state.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: reconciliation-demo
data:
  status: "active"
```

Apply it: `kubectl apply -f demo.yaml`

Step 2: The Manual "Reconcile" Loop

We will simulate the controller's logic in a bash loop. Open a terminal and run the following command to mimic an asynchronous controller watching for changes:

```
while true; do
  echo "--- Checking state at $(date) ---"
  # Observe
  CURRENT=$(kubectl get cm reconciliation-demo -o jsonpath='{.data.status}')

  # Analyze
  if [ "$CURRENT" != "active" ]; then
    echo "Drift detected! Fixing..."
    # Act
    kubectl patch cm reconciliation-demo -p '{"data": {"status": "active"}}'
  else
    echo "State consistent."
  fi

  sleep 5
done
```

Step 3: Triggering a Discrepancy

Open a second terminal and modify the resource manually: `kubectl patch cm reconciliation-demo -p '{"data": {"status": "broken"}}'`

Watch the first terminal. Within 5 seconds, the "controller" will detect the drift (the current state no longer matching the desired state) and reconcile it back to "active".

Troubleshooting Guidance

- **Infinite Requeue Loops:** If your logic fails to reach the desired state, ensure you are not queuing the same item indefinitely without backoff, as this will spike CPU usage.
- **Eventual Consistency:** Remember that because the Informer cache is updated asynchronously, there may be a slight delay between a change in the API server and the controller's awareness

of that change. Always design your system to handle "eventual" rather than "instant" convergence.

Week 1.1 — Kubernetes Resource Model

Week 1.1 — Kubernetes Resource Model

The Kubernetes Resource Model centers on declarative configuration management, where users define the desired state of the system rather than the steps to achieve it. This module explores the foundational mechanics of how the `kube-apiserver` processes these intent-based requests.

Key concepts include:

- **Declarative Intent:** The core principle of defining the target state, which the system continuously reconciles.
- **Request Lifecycle:** A deep dive into the plugin layers within the `kube-apiserver`, including authentication, authorization, audit, and admission control.
- **Write Operations:** Understanding how mutation and validation webhooks intercept `CREATE` and `UPDATE` calls to enforce policies or modify objects.
- **Control Plane Scaling:** Emphasis on vertical scaling strategies for `etcd` and the `kube-apiserver` to ensure cluster stability.

The declarative intent of Kubernetes Resources

The Declarative Intent of Kubernetes Resources

In traditional systems administration, operators often use **imperative** commands (e.g., "Install this package," "Start this service," "Restart this process"). In Kubernetes, this model is inverted. Kubernetes is fundamentally a **declarative, API-centric configuration management system**.

The Core Philosophy: Desired vs. Actual State

The "declarative intent" refers to the practice of describing the **goal** (the "what") rather than the **steps** (the "how"). You provide the Kubernetes API with a blueprint—the desired state—and the system works tirelessly to make the actual state match that description.

Imperative vs. Declarative

- **Imperative:** You give a sequence of instructions. If a step fails halfway through, the system is left in an inconsistent, partial state. You must manually account for every possible failure scenario.
- **Declarative:** You define the final result in a YAML or JSON manifest. If the system crashes, is updated, or is tampered with, the control plane identifies the drift between the current state and the declared state and performs the necessary actions to reconcile them.

Anatomy of Declarative Intent

Every Kubernetes object relies on a standard structure defined in its manifest. This structure ensures that the API server can process and store your intent effectively.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec: ❶
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template: ❷
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
```

- ❶ **Spec:** The definitive record of the "Desired State." This is what you, the user, want the system to look like.
- ❷ **Template:** A blueprint used by controllers to create new resources (like Pods) that conform to your intent.

How the System Processes Intent

The transition from intent to reality happens through a continuous, asynchronous reconciliation loop:

1. **Submission:** You submit a resource manifest to the `kube-apiserver`.
2. **Validation:** The API server ensures the manifest is syntactically and semantically correct (via Admission Webhooks).
3. **Persistence:** The intent is saved as a structured object in `etcd`.
4. **Observation:** Controllers (like the Deployment Controller) watch the API server for changes.
5. **Reconciliation:** The controller compares the current state (e.g., "0 pods are running") with your desired state ("3 pods should be running").
6. **Action:** The controller issues commands to align reality with your intent.

Hands-on Activity: Observing Drift and Self-Healing

In this activity, we will demonstrate how Kubernetes enforces your declarative intent despite external interference.

Prerequisites

- A running Kubernetes cluster (e.g., Minikube, Kind, or OpenShift).
- `kubectl` configured and authenticated.

Step 1: Create the Desired State

Apply a simple deployment manifest to the cluster.

```
kubectl create deployment nginx-demo --image=nginx --replicas=2
```

Step 2: Observe the System Action

Check that the deployment has reconciled the desired state.

```
kubectl get pods -l app=nginx-demo  
# You should see 2 pods running.
```

Step 3: Introduce "Drift" (Imperative Interference)

Manually delete one of the pods. This represents a system failure or accidental deletion.

```
# Find a pod name  
POD_NAME=$(kubectl get pods -l app=nginx-demo -o name | head -n 1)  
# Delete it  
kubectl delete $POD_NAME
```

Step 4: Validate Self-Healing

Query the pods again immediately.

```
kubectl get pods -l app=nginx-demo
```

Observation: You will notice that a new pod is being created almost instantly. The Deployment controller observed that the **actual** state (1 pod) no longer matched the **declared** state (2 pods) and automatically rectified the situation.

Key Takeaway: You never told the system to "create a new pod." You told the system to maintain 2 replicas, and the system took ownership of that intent.

kube-apiserver plugin request/response lifecycle

Understanding the **kube-apiserver** Request/Response Lifecycle

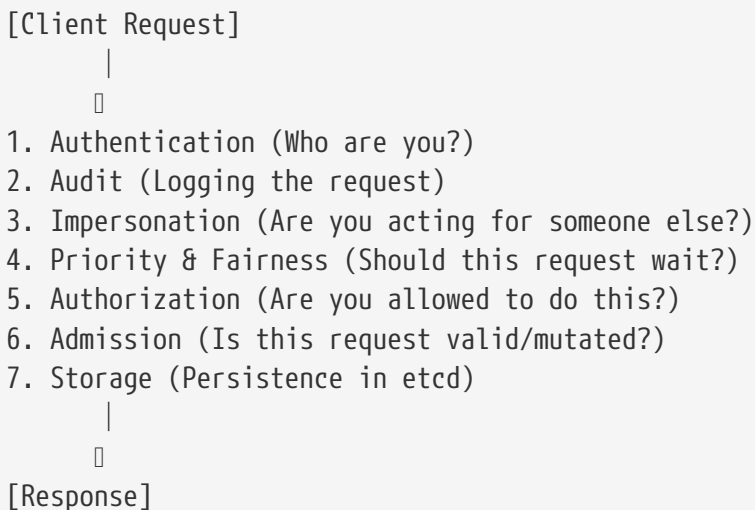
The **kube-apiserver** serves as the central nervous system of a Kubernetes cluster. Every interaction—whether from a **kubectl** command, an internal controller, or an external operator—passes through the API server's request pipeline. Understanding this lifecycle is critical

for building secure, efficient, and extension-ready Kubernetes applications.

The Plugin Pipeline

When a request enters the `kube-apiserver`, it does not immediately interact with the storage layer (`etcd`). Instead, it traverses a sophisticated chain of plugins. These plugins act as filters, ensuring that the request is legitimate, authorized, and compliant with cluster policies.

The lifecycle follows this logical flow:



Step-by-Step Breakdown

- **Authentication:** The server verifies the identity of the requester using certificates, tokens, or OIDC. If authentication fails, the request is rejected with a `401 Unauthorized`.
- **Audit:** Every request is logged to track cluster activity. This is essential for compliance and security auditing.
- **Impersonation:** Enables admins to act on behalf of another user. The plugin checks if the caller has the authority to impersonate the target user.
- **Priority & Fairness (APF):** Prevents the API server from being overwhelmed. It categorizes requests (e.g., leader election vs. background listing) and applies rate limits to ensure cluster stability.
- **Authorization:** Checks if the authenticated user has the RBAC permissions to perform the requested verb (GET, POST, DELETE) on the specific resource.
- **Admission:** This is the most extensible layer.
- **Mutating:** Modify the object (e.g., injecting sidecars).
- **Validating:** Enforce business logic or security constraints (e.g., rejecting pods that run as root).
- **Storage:** If all plugins pass, the state is serialized and written to `etcd`.

Practical Exploration: Investigating API Discovery

To understand how the API server presents its capabilities, we can inspect the API discovery endpoint. This is a great way to see how the server negotiates content types and versions.



Ensure you have `oc` (or `kubectl`) configured to access your cluster, and use `oc proxy` to securely reach the API server.

Hands-on Activity: Probing the API Server

1. Start the proxy:

```
oc proxy --port=8001 &
```

2. Query the API discovery endpoint:

Use the following command to retrieve the discovery information. The specific headers are required to get the current `v2` discovery format.

```
curl -v http://127.0.0.1:8001/api \
-H "Accept: application/json;v=v2;g=apidiscovery.k8s.io;as=APIGroupDiscoveryList"
```

3. Analyze the response:

You will receive a JSON payload detailing the resources available in the core API group. This response confirms that your request successfully passed the authentication and authorization filters defined in the lifecycle above.

Key Takeaways for Developers

- **Order Matters:** Admission webhooks occur **after** authentication and authorization. You don't need to check user identity inside a webhook; the `kube-apiserver` has already validated the user.
- **Performance Impact:** Excessive complexity in Mutating or Validating webhooks adds latency to every single API operation. Keep your webhooks lean.
- **Observability:** If your requests are being rejected, check your RBAC (Authorization) first, followed by your Admission Webhook configurations.

Week 1.2 — Controllers, informers, client-go

Week 1.2 — Controllers, informers, client-go

This module covers the fundamental building blocks of Kubernetes operators and resource synchronization. It focuses on the mechanics of how custom controllers observe state and ensure the cluster converges toward a desired configuration.

Key components include:

- **client-go Architecture:** The official Go client library for interacting with the Kubernetes API.
- **Informers:** Act as the "eyes" of the controller, watching for cluster events and maintaining a local cache to minimize API server load.
- **Listers:** Provide read-only access to the local cache, allowing controllers to retrieve resource states efficiently.
- **Workqueues:** Serve as buffers for processing events; they implement FIFO (First-In-First-Out) logic to ensure reliable task management.
- **Event Propagation:** Understanding how changes in the API server flow through caches to trigger controller logic.
- **Operational Patterns:** Best practices for error handling, effective requeue strategies, and managing concurrency within reconciliation loops.

client-go architecture fundamentals

Week 1.2: client-go Architecture Fundamentals

The `client-go` library is the official Go client for Kubernetes and serves as the foundation for virtually all Kubernetes controllers and operators. Understanding its architecture is critical for building resilient, high-performance automation that interacts with the Kubernetes API.

The Role of client-go

In a distributed system like Kubernetes, the controller needs to observe the state of the cluster, react to changes, and move the cluster toward a desired state. `client-go` provides the mechanisms to perform these tasks efficiently without overloading the `kube-apiserver`.

Key components of `client-go` include:

- **Clients:** Tools to interact with the API (e.g., `kubernetes.Clientset`, `dynamic.Interface`).
- **Informers:** Mechanisms to watch for changes and maintain a local cache.
- **Listers:** Tools to query the local cache.
- **Workqueues:** Buffers that hold keys for processing, ensuring thread-safe, serialized reconciliation.

1. The Informer Pattern

Instead of polling the API server (which is inefficient and doesn't scale), `client-go` uses the **Informer** pattern. An Informer maintains a local cache of objects and provides a notification system via event handlers (Add, Update, Delete).

- **Reflector:** A tool that watches the Kubernetes API for a specific resource type and lists/watches for changes.
- **DeltaFIFO:** The Reflector puts all changes into a queue called `DeltaFIFO`. This is a First-In-First-Out queue that tracks the "delta" (the change) between the current state and the observed state.
- **Indexer:** The local cache that stores the objects, allowing for fast lookups by namespace, labels, or other attributes.

2. Listers

A Lister is a read-only interface used to interact with the Indexer (local cache). By using a Lister instead of a direct API call, your controller avoids constant network overhead, significantly reducing the load on the `kube-apiserver`.

3. Workqueues

The Informer detects an event and notifies the controller. The controller does not typically process the object immediately in the callback; instead, it adds the object's key (e.g., `namespace/name`) to a **Workqueue**. * **Serialized Processing:** Workqueues ensure that even if multiple events occur for the same resource, the controller handles them in an orderly, serialized fashion. * **Retries:** The queue supports "requeueing" logic, which is essential for handling transient errors during the reconciliation process.

Architectural Flow

The high-level interaction follows this pipeline: `API Server` → `Reflector` → `DeltaFIFO` → `Informer (Indexer)` → `Event Handler` → `Workqueue` → `Controller Logic`.



Remember that Workqueues operate on a **FIFO (First-In-First-Out)** basis. This ensures that the controller processes events in the order they were observed, maintaining the integrity of the state reconciliation.

Hands-on Activity: Observing the Informer

While we will build a full controller later, you can observe the informer lifecycle by inspecting the `client-go` cache initialization.

Initialize a client and informer factory in your Go project:

```
import (
```

```

    "k8s.io/client-go/informers"
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/clientcmd"
)

// Initialize the clientset
clientset, _ := kubernetes.NewForConfig(config)

// Create a shared informer factory
// The '0' here indicates no resync period
factory := informers.NewSharedInformerFactory(clientset, 0)

// Get the pod informer
podInformer := factory.Core().V1().Pods().Informer()

// Add event handlers
podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: func(obj interface{}) {
        fmt.Println("Pod added!")
    },
})

```

Discussion Points

- **Why Caches Exist:** Direct API requests are slow and put pressure on etcd. Local caches allow for $O(1)$ or $O(\log n)$ lookups.
- **Event Propagation:** If a pod is deleted, the Reflector receives an event, updates the Indexer, and the Informer triggers the `DeleteFunc` callback, placing the key in the workqueue for the controller to handle.

Troubleshooting Tips

- **Stale Caches:** Ensure your Informer is fully synchronized (`cache.WaitForCacheSync`) before starting your worker routines, or you may try to process items that the cache doesn't know about yet.
- **Queue Blocking:** If your worker logic takes too long, it blocks the queue. Always use multiple workers if the processing logic involves long-running network calls.

Informers, listers, and workqueues

Week 1.2: Informers, Listers, and Workqueues

In the Kubernetes controller pattern, the goal is to drive the current state of the system toward a desired state. To do this efficiently without overwhelming the `kube-apiserver`, we rely on `client-go` components that optimize communication and event processing.

The Controller Architectural Pattern

The `client-go` library provides the plumbing required to build high-performance controllers. At its heart, the architecture is designed to handle asynchronous event propagation.

The Role of Components

You can visualize the data flow as a pipeline:

- **Informers:** Acting as the "eyes" of the controller, Informers watch the API server for resource changes. They use long-polling (or HTTP/2 streaming) to receive events (Add, Update, Delete) and keep a local state.
- **Listers:** These provide a read-only interface to the local cache maintained by the Informer. Instead of querying the `kube-apiserver` every time you need to check an object, you query the local cache via the Lister, significantly reducing latency and API server load.
- **Workqueues:** These act as buffers for event processing. When an Informer detects a change, it pushes the **key** (namespace/name) of the affected object into a Workqueue. The controller then picks up these keys to perform reconciliation.



Queues are strictly FIFO. Unlike a stack (LIFO), a workqueue ensures that the oldest events are processed first, maintaining sequential integrity for state changes.

How Data Flows: The Synchronization Loop

1. **Reflector:** The Reflector watches the API server for a specific resource type. When it detects an event, it places the object into a `DeltaFIFO` queue.
2. **Informer:** The Informer processes items from the `DeltaFIFO` queue, updates the local cache (Store), and triggers registered Event Handlers.
3. **Event Handler:** Usually, the handler simply takes the object's key and adds it to the **Workqueue**.
4. **Worker:** The controller's `processNextWorkItem` function pulls the key from the Workqueue and executes the `Reconcile` logic.

Hands-On Activity: Analyzing Queue Behavior

In this exercise, you will observe how a `workqueue` handles multiple updates to the same object.

Prerequisites

- Go installed (1.21+)
- `k8s.io/client-go` module initialized in your project

Steps

1. **Initialize the Workqueue:** Inside your controller logic, create a new rate-limiting queue:
[source,go] ---- queue :=

```
workqueue.NewNamedRateLimitingQueue(workqueue.DefaultControllerRateLimiter(), "my-controller") ----
```

2. **Enqueue a Key:** In your Informer's `AddFunc` or `UpdateFunc`, extract the key: [source,go] ----
`key, err := cache.MetaNamespaceKeyFunc(obj) if err == nil { queue.Add(key) } ----`
3. **Implement the Worker:** Create a loop that pulls items and processes them. [source,go] ----
`func (c *Controller) processNextItem() bool { key, quit := c.queue.Get() if quit { return false } defer c.queue.Done(key)`

```
// Use the Lister to get the object from local cache
obj, exists, _ := c.lister.GetByKey(key.(string))
```

```
// ... perform reconciliation logic ...
```

```
    return true
}
----
```

4. **Error Handling and Requeueing:** If your reconciliation fails, use `queue.AddRateLimited(key)` to put the item back into the queue. The `RateLimiter` will implement exponential backoff, preventing your controller from entering a hot-loop if an error persists.

Troubleshooting Gotchas

- **Cache Inconsistency:** If your controller reads from a `Lister` but the cache hasn't synced yet (e.g., during controller startup), you will get empty results. Always ensure `cache.WaitForCacheSync` is called before starting your workers.
- **Blocking the Queue:** Never perform long-running blocking operations (like calling external APIs) directly inside the `Reconcile` loop. If the worker blocks, the queue depth will grow, and your controller will fall behind the cluster state.
- **Key Overwriting:** If a resource is updated rapidly, the queue only needs to store the key. The worker will always fetch the **latest** version from the cache when it eventually processes the key, effectively "collapsing" intermediate states.

Cache mechanisms and event propagation

Cache Mechanisms and Event Propagation

In Kubernetes, controllers do not query the `kube-apiserver` for every single operation. Doing so would overwhelm the API server and introduce significant latency. Instead, `client-go` provides a robust caching and synchronization mechanism that allows controllers to operate on a local, near-real-time replica of the cluster state.

The Role of Local Caching

The primary goal of a cache in `client-go` is to decouple the controller's logic from the API server's throughput. By maintaining a local copy of objects, the controller can:

- * **Reduce API Load:** Read operations (List/Get) are served from memory, not the network.
- * **Improve Performance:** Local lookups via Listers are significantly faster than HTTP requests.
- * **Handle Disconnects:** If the network to the `kube-apiserver` flickers, the controller can continue processing its current queue state.

Event Propagation: From etcd to Controller

Event propagation is the pipeline that ensures the local cache remains synchronized with the ground truth in `etcd`.

```
@startuml
skinparam componentStyle uml2
[etcd] -> [kube-apiserver] : Watch Event
[kube-apiserver] -> [Reflector] : HTTP/Watch
[Reflector] -> [DeltaFIFO] : Add/Update/Delete
[DeltaFIFO] -> [Indexer] : Update Cache
[Indexer] -> [Informer] : Notify Event Handler
@enduml
```

1. The Reflector

The `Reflector` is the foundation. It performs a `List` and `Watch` operation on a specific resource type. It populates the initial state and then watches for subsequent changes (ADD, UPDATE, DELETE). These changes are pushed into the `DeltaFIFO` queue.

2. DeltaFIFO: The Buffer

The `DeltaFIFO` is a queue that stores the changes (deltas) received from the `Reflector`.

- * **Crucial Note:** Queues in Kubernetes are **strictly FIFO** (First-In, First-Out).
- * It ensures that even if events arrive in rapid succession, the controller processes them in the order they occurred.

3. The Indexer (Local Store)

The `Indexer` is the component that holds the actual state. It acts as the local database for the controller. When an item is pulled from the `DeltaFIFO`, the `Indexer` is updated to reflect the new state, ensuring that the controller's `Lister` has access to the most recent object version.

4. Informer Event Handlers

Once the cache is updated, the `Informer` triggers user-defined event handlers (`AddFunc`, `UpdateFunc`, `DeleteFunc`). These handlers typically do not perform business logic directly; instead, they add the **key** (namespace/name) of the object to a `WorkQueue`.

Practical Exercise: Inspecting Cache Synchronization

In this exercise, you will observe how the cache handles an event.

Steps

1. **Initialize an Informer:** Using `client-go`, create a `SharedInformerFactory` for Pods. [source,go]
---- `factory := informers.NewSharedInformerFactory(clientset, time.Second*30)` `podInformer := factory.Core().V1().Pods().Informer()` ----
2. **Add Event Handlers:** Log the events to see the propagation in action. [source,go]
---- `podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{ AddFunc: func(obj interface{}) { pod := obj.(*corev1.Pod) fmt.Printf("Cache Updated: Pod %s added\n", pod.Name) }, })` ----
3. **Start the Informer:** Run the informer in a background goroutine. [source,go]
---- `stopCh := make(chan struct{}) defer close(stopCh) factory.Start(stopCh) cache.WaitForCacheSync(stopCh, podInformer.HasSynced)` ----
4. **Observe:** Create a Pod in your cluster using `kubectl run`. Watch your application logs to see the `AddFunc` trigger immediately after the cache syncs the event.

Key Takeaways

- **Informers** are your "eyes" on the cluster.
- **Listers** are your read-only view of the local cache.
- **WorkQueues** are your buffers for processing items asynchronously.
- **Always use FIFO:** Never attempt to reorder events; consistency is maintained by processing the stream of changes in the order they were emitted by the API server.

Error handling and requeue strategies

Week 1.2: Error Handling and Requeue Strategies in Controllers

In a distributed system like Kubernetes, a controller cannot assume that every operation will succeed on the first attempt. Network partitions, transient API errors, or conflicting states are expected realities. Robust controllers must implement sophisticated error handling and requeue strategies to ensure **eventual consistency**.

The Anatomy of a Requeue

When a `Reconcile` function encounters an error, it should not simply crash the process. Instead, it must signal to the controller's workqueue that the item failed and requires processing again later.

The workqueue acts as a buffer between the `Informer` (which watches for changes) and the `Reconciler` (which acts on them). When processing an item fails, the standard pattern involves:

1. **Logging the error:** Provide context for observability.
2. **Evaluating the error type:** Determine if the error is transient (e.g., API server timeout) or

permanent (e.g., invalid resource configuration).

3. **Re-adding to the queue:** Using a rate-limiting mechanism to avoid hammering the API server during recovery.

Rate Limiting and Backoff

If a controller fails to process a resource, retrying immediately can exacerbate issues (a "thundering herd" effect). We use rate limiters to implement exponential backoff:

- **Linear Backoff:** Retrying at fixed intervals.
- **Exponential Backoff:** Increasing the delay between retries (e.g., 1s, 2s, 4s, 8s...).



In `client-go` and `controller-runtime`, never use `time.Sleep()` inside your reconcile loop. This blocks the entire worker thread. Always return an error to the reconciler, which then utilizes the workqueue's `AddRateLimited()` method.

Hands-on Activity: Implementing Resilient Requeues

This activity demonstrates how to handle errors within a reconcile loop using `controller-runtime` patterns.

Prerequisites

- A scaffolded project using `kubebuilder` or a basic controller setup.
- Access to a Kubernetes cluster (or `kind`).

Step 1: Update the Reconcile Logic

Modify your controller's `Reconcile` function to return an error when your logic fails.

```
func (r *MyReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    // 1. Fetch the object
    obj := &myappv1.MyResource{}
    if err := r.Get(ctx, req.NamespacedName, obj); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    // 2. Perform business logic
    if err := r.doSomething(obj); err != nil {
        // Log the error and return it to trigger a requeue
        log.Log.Error(err, "Failed to reconcile MyResource")
        return ctrl.Result{}, err
    }

    return ctrl.Result{}, nil
}
```

Step 2: Customizing Requeue Intervals

Sometimes you don't have an error, but you need to check the status again after a specific duration (e.g., waiting for a dependent resource to become ready).

```
import "time"

// Inside Reconcile...
if !obj.Status.Ready {
    // Requeue after 30 seconds to check again
    return ctrl.Result{RequeueAfter: time.Second * 30}, nil
}
```

Step 3: Best Practices for Error Handling

- **IgnoreNotFound:** Always use `client.IgnoreNotFound(err)` when fetching objects. If the object is deleted while in the queue, you should stop processing, not return an error.
- **Don't Requeue Invalid Configs:** If a user provides an invalid spec (e.g., a field that fails validation), retrying will never succeed. Update the object's `Status` to reflect a "Failed" condition and stop returning the error.
- **Use Conditions:** Represent the state of the controller in the `Status` field using `metav1.Condition`. This allows users to see **why** the controller is stuck without checking logs.

Summary

- **Queues are FIFO:** Remember that order matters; ensure your logic is idempotent.
- **Never Block:** Use `ctrl.Result{RequeueAfter: ...}` instead of sleeping.
- **Rate Limit:** Allow your controller to back off from failing resources to protect the `kube-apiserver`.

Week 2.1 — Controllers contd.

Week 2.1 — Controllers contd.

This module focuses on developing robust Kubernetes controllers using the `controller-runtime` abstraction and `kubebuilder` framework. It emphasizes the practical implementation of the reconciliation loop to manage the lifecycle of custom resources.

Key topics include:

- **Spec vs. Status:** Managing the delta between desired state (Spec) and actual state (Status) via conditions.
- **API Lifecycle:** Implementing comprehensive CRUD operations using `get`, `list`, `watch`, `create`, `delete`, `update`, and `patch` semantics.
- **Best Practices:** Differentiating between `Apply`, `Patch`, and `Update` operations.
- **Validation:** Ensuring API integrity using `kube-api-linter` (KAL) and built-in validation rules.
- **Tooling:** Using `kubebuilder` to scaffold and streamline the development of production-ready controllers.

Reconcile loop and controller-runtime abstraction

Reconcile Loop and controller-runtime Abstraction

In Kubernetes, the control plane is a collection of "control loops." Instead of imperative commands, Kubernetes relies on a **declarative model** where controllers continuously work to drive the actual state of the system toward the desired state defined in the object's specification.

The Reconcile Loop Explained

The heart of any Kubernetes controller is the **Reconcile Loop**. It is a control loop that constantly observes the current state of a resource and performs the necessary actions to achieve the desired state.

The reconcile process follows three core steps:

1. **Observe:** Retrieve the current state of the resource from the Kubernetes API server (often via a cached lister).
2. **Analyze:** Compare the current state (the `Status`) against the desired state (the `Spec`).
3. **Act:** Perform operations (create, update, delete, or patch) to synchronize the system to match the `Spec`.

The controller-runtime Abstraction

While writing controllers using raw `client-go` is possible, it involves significant boilerplate (handling informers, workqueues, and cache synchronization). `controller-runtime` is the standard library used by the community to abstract these complexities.

Key benefits of `controller-runtime` include: * **Manager**: Orchestrates all controllers in the process, handling shared caches and leader election. * **Reconciler Interface**: Provides a simple `Reconcile(ctx, request)` signature that hides the underlying queue management. * **Event Filters**: Allows you to filter events (e.g., ignore status updates) to prevent unnecessary reconciliation cycles.

Spec vs. Status: The Truth of State

Understanding the separation of concerns between `Spec` and `Status` is critical for building idempotent controllers:

- **Spec (Desired State)**: The definition of what the user wants. The controller must treat this as "read-only" and authoritative.
- **Status (Actual State)**: A reflection of the current reality. The controller is responsible for updating this field based on the work it performs.



Never modify the `Spec` from within your controller's reconcile loop. Modifying the `Spec` causes an infinite reconciliation loop, as the controller will detect a change in the desired state and trigger itself again.

Hands-on Activity: Defining the Reconciler

In this activity, we will define a basic reconciler structure using `controller-runtime`.

Prerequisites

- Go installed (version 1.21+)
- `kubebuilder` initialized project

Step 1: Define the Reconciler Struct

In your `internal/controller/` directory, define your reconciler.

```
type MyResourceReconciler struct {
    client.Client
    Scheme *runtime.Scheme
}

// Reconcile is the main logic loop
func (r *MyResourceReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    // 1. Fetch the object
    var myObj myv1.MyResource
    if err := r.Get(ctx, req.NamespacedName, &myObj); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    // 2. Add your logic here (e.g., checking Spec vs Status)

    // 3. Update the Status
}
```

```
    return ctrl.Result{}, nil
}
```

Step 2: Set up the Manager

Register your controller with the Manager in `main.go`. The manager handles the orchestration of the cache and ensures the Reconcile loop is triggered whenever the watched resource changes.

```
if err = (&controller.MyResourceReconciler{
    Client: mgr.GetClient(),
    Scheme: mgr.GetScheme(),
}).SetupWithManager(mgr); err != nil {
    os.Exit(1)
}
```

Step 3: Implement Idempotency

To ensure your controller is safe, always ensure your logic is idempotent: * **Check before creating:** Use `client.Get` to see if a resource exists before attempting a `client.Create`. * **Use Patch over Update:** `Update` performs a full object overwrite, which is prone to conflicts. `Patch` allows for surgical updates to specific fields, reducing the likelihood of resource version conflicts.

Troubleshooting Tips

- **Requeueing:** If a resource is not yet ready, return `ctrl.Result{RequeueAfter: time.Second * 5}, nil` to trigger a re-check later rather than failing the reconcile.
- **Logging:** Use `log.FromContext(ctx)` provided by `controller-runtime` to keep consistent audit logs across your control plane.

Scaffolding with kubebuilder

Scaffolding with Kubebuilder

In the Kubernetes ecosystem, the manual construction of Custom Resource Definitions (CRDs) and the associated controller logic is error-prone. **Kubebuilder** is the industry-standard framework for building Kubernetes APIs using Custom Resource Definitions (CRDs). It provides a scaffolding tool that generates the boilerplate code required to run a controller, allowing developers to focus on business logic rather than wiring the `client-go` internals.

Why Kubebuilder?

Kubebuilder leverages `controller-runtime`, a set of libraries that abstract the complexities of the Kubernetes API. By using the scaffolding CLI, you ensure your project follows the layout patterns expected by the community.

- **Standardized Structure:** Consistent project layout for Go-based operators.
- **Controller-Runtime Integration:** Built-in support for reconciliation loops, caching, and event

handling.

- **Built-in Testing:** Scaffolds `envtest` for spinning up local control planes during testing.
- **Lifecycle Management:** Simplified creation of Webhooks, CRDs, and RBAC manifests.

Scaffolding a New Project

Before you begin, ensure you have `kubebuilder` installed and your `GOPATH` configured.

Step 1: Initialize the Project

Initialize your Go module and scaffold the base project structure.

```
mkdir my-operator
cd my-operator
kubebuilder init --domain example.com --repo github.com/example/my-operator
```

- `--domain`: Used for group names in your API groups.
- `--repo`: The Go module name for your project.

Step 2: Create an API

To create a Custom Resource (CR), you must define its Group, Version, and Kind (GVK).

```
kubebuilder create api --group webapp --version v1 --kind Guestbook
```

This command creates the following directory structure: * `api/v1/guestbook_types.go`: Where you define the `Spec` (desired state) and `Status` (actual state) of your resource. * `internal/controller/guestbook_controller.go`: The scaffolded controller where you implement the `Reconcile` loop.

Deep Dive: The Reconcile Loop

The heart of your controller is the `Reconcile` function. Kubebuilder scaffolds this logic for you:

```
func (r *GuestbookReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    // 1. Fetch the object
    // 2. Implement business logic
    // 3. Update Status if necessary
    // 4. Return result (requeue if needed)
    return ctrl.Result{}, nil
}
```


Hands-on Activity: Defining the Spec and Status

1. **Open** `api/v1/guestbook_types.go`.
2. **Define your Schema:** Add fields to `GuestbookSpec` (e.g., `Replicas int`) and `GuestbookStatus` (e.g., `Ready bool`).
3. **Generate Manifests:** Run the following command to update your CRD manifests based on your Go struct changes:

```
make manifests
```

Best Practices & Gotchas

- **API Conventions:** Always refer to the [official Kubernetes API Conventions](<https://github.com/kubernetes/community/blob/master/contributors/devel/sig-architecture/api-conventions.md>) when naming your fields.
- **Validation:** Use `kubebuilder` markers (e.g., `// +kubebuilder:validation:Minimum=1`) to enforce schema constraints at the API level.
- **Use `make` targets:** `Kubebuilder` generates a `Makefile`. Utilize `make install` to install CRDs into your cluster and `make run` to execute the controller locally against your current `KUBECONFIG`.
- **Linting:** Integrate `kube-api-linter` (KAL) into your CI pipeline to ensure your API structure adheres to Kubernetes best practices before submission.

Summary

Scaffolding with `Kubebuilder` accelerates the path from "idea" to "running controller." It abstracts the tedious `client-go` boilerplate, providing a clean, production-ready environment that prioritizes the declarative reconciliation pattern essential for distributed system management.

Spec vs Status and condition management

Spec vs. Status and Condition Management

In the Kubernetes Resource Model, the separation of concerns between what a user wants and what the system is currently doing is fundamental to the declarative nature of the platform. Understanding the relationship between `spec`, `status`, and `conditions` is critical for building robust, idempotent controllers.

The Core Distinction

Spec (The Desired State)

The `spec` defines the **intent**. It is the source of truth provided by the user (via YAML/JSON) or another controller. * **Immutability:** While not strictly enforced by the schema, the `spec` should generally be treated as read-only by the controller. * **Controller's Role:** Your controller monitors the `spec` and works to ensure the actual state of the cluster matches this definition.

Status (The Observed State)

The **status** represents the **observed state** of the resource. It is the controller's feedback loop to the user. * **Source of Truth:** Only the controller (or the system) should update the **status**. * **Transient Nature:** If a controller is deleted, the **spec** remains, but the **status** may become stale. It is intended to reflect the result of the controller's reconciliation efforts.

Condition Management

Conditions provide a structured, machine-readable way to report the progress and health of a resource. Rather than just a raw string, a **Condition** allows users to understand **why** a resource might not be ready.

A typical **Condition** structure includes: * **Type:** The specific state being reported (e.g., **Available**, **Progressing**, **Degraded**). * **Status:** The current state of that type (**True**, **False**, or **Unknown**). * **Reason:** A CamelCase machine-readable identifier for the current state. * **Message:** A human-readable explanation of the state. * **ObservedGeneration:** A critical field that links the condition to a specific version of the **spec**.



Always compare **metadata.generation** with **status.conditions[].observedGeneration**. If they do not match, the controller is still processing a change, and the status does not yet reflect the current **spec**.

Hands-on Activity: Implementing Status Conditions

In this activity, we will use the **controller-runtime** library to update a status condition.

Prerequisites

- A project scaffolded with **kubebuilder**.
- A defined API struct with a **status** field.

Step 1: Update the Status Struct

Ensure your **types.go** file includes a slice of **metav1.Condition**.

```
type MyResourceStatus struct {
    // +patchMergeKey=type
    // +patchStrategy=merge
    // +listType=map
    // +listMapKey=type
    Conditions []metav1.Condition `json:"conditions,omitempty"`
}
```

Step 2: Implement the Reconcile Logic

In your **Reconcile** function, use the **meta** library to update the conditions cleanly.

```
import (
```

```

    "k8s.io/apimachinery/pkg/api/meta"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

func (r *MyReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result,
error) {
    // 1. Fetch the object
    // 2. Perform logic...

    // 3. Update Condition if ready
    condition := metav1.Condition{
        Type:          "Ready",
        Status:        metav1.ConditionTrue,
        Reason:        "ReconciliationSucceeded",
        Message:       "The resource is fully operational",
        ObservedGeneration: myObj.Generation,
    }

    meta.SetStatusCondition(&myObj.Status.Conditions, condition)

    // 4. Persist changes to the API server
    err := r.Status().Update(ctx, &myObj)
    return ctrl.Result{}, err
}

```

Gotchas and Best Practices

- **Avoid "Status-in-Spec":** Never mirror status information back into the `spec`. This creates infinite loops where the controller triggers its own reconciliation.
- **Patch vs. Update:** Use `Status().Update()` for simple changes. If you are dealing with high-concurrency environments or complex field merging, consider `Status().Patch()`.
- **Idempotency:** Your reconciliation logic must be idempotent. Every time the loop runs, calculate the **desired** status based on the current world state, then apply it. Do not "increment" values in status; "set" them.
- **Generation Tracking:** Always check if the resource has been updated by comparing `metadata.generation` to your last recorded `observedGeneration` to prevent unnecessary status updates.

API validation and linting

API Validation and Linting

In the Kubernetes Resource Model, ensuring the integrity and quality of your Custom Resource Definitions (CRDs) is paramount. Because Kubernetes relies on declarative state, "bad" API definitions can lead to fragmented control planes, broken reconciliation loops, and unpredictable cluster behavior.

API validation and linting ensure that your CRDs adhere to community standards, follow best practices for API design, and minimize runtime errors.

Why Validation and Linting Matter

1. **Semantic Consistency:** Ensuring that `Spec` fields follow standard naming conventions (e.g., camelCase) and type definitions.
2. **Structural Integrity:** Verifying that required fields are present and that optional fields have appropriate default values.
3. **Performance:** Well-defined schemas allow the `kube-apiserver` to perform efficient validation without needing to invoke external webhooks for simple type checking.
4. **Developer Experience:** Catching errors at development time (CI/CD) rather than at runtime when a user attempts to apply a YAML manifest.

Common Validation Approaches

1. OpenAPIV3Schema

The primary method for validating Kubernetes resources is through the `validation` block in your CRD manifest. By defining an `OpenAPIV3Schema`, you provide the `kube-apiserver` with the constraints necessary to reject invalid requests before they reach your controller.

```
spec:
  validation:
    openAPIV3Schema:
      type: object
      properties:
        spec:
          type: object
          properties:
            replicas:
              type: integer
              minimum: 1
              maximum: 10
            image:
              type: string
              pattern: '^registry\\.example\\.com/.*'

```

2. Kube-API-Linter (KAL)

The Kubernetes API Linter (KAL) acts as a static analysis tool for your API definitions. It checks your CRD Go structs or manifests against a set of rules (e.g., `names-must-match`, `no-empty-fields`).

- **Rule Enforcement:** Ensures that API groups follow the `v1alpha1` → `v1beta1` → `v1` progression.
- **Best Practices:** Flags common mistakes, such as missing `json` tags in Go structs or missing `DeepCopy` methods.

Hands-on Activity: Integrating Linting in Kubebuilder

In this activity, we will integrate linting practices into your `kubebuilder` workflow to ensure your API remains clean and production-ready.

Prerequisites

- Ensure `kubebuilder` is installed.
- An existing project scaffolded with `kubebuilder`.

Step 1: Annotating Go Structs

Validation is not just about YAML; it is about Go code. Use `controller-gen` markers to enforce validation rules directly in your Go source code.

Open your `api/v1/myresource_types.go` file and add validation markers:

```
type MyResourceSpec struct {
    // +kubebuilder:validation:Minimum=1
    // +kubebuilder:validation:Maximum=10
    Replicas int32 `json:"replicas,omitempty"`

    // +kubebuilder:validation:Pattern=`^registry\.example\.com/.*\`
    Image string `json:"image,omitempty"`
}
```

Step 2: Running the Linting Pipeline

Before building your manifests, run the generation tool to verify your changes:

```
# Run controller-gen to update CRD manifests with the validation markers
make manifests

# Verify that the generated config/crd/bases file contains the
# OpenAPIV3Schema constraints you defined in Go.
cat config/crd/bases/mygroup.example.com_myresources.yaml
```

Step 3: Best Practice Tip

Always integrate `make manifests` into your CI pipeline. If a developer attempts to submit a PR that changes the API without satisfying the linting/validation criteria, the pipeline should fail before the code is ever merged into the main branch.

Summary

While `OpenAPIV3Schema` handles runtime validation at the `kube-apiserver` level, tools like `kube-api-linter` ensure your codebase remains maintainable. Use these tools in tandem to shift left your validation process, catching API design flaws before they reach the cluster.

CRUD operations and apply/patch/update semantics

CRUD Operations and Apply/Patch/Update Semantics

In the Kubernetes control plane, the way we modify state is critical. Because Kubernetes is an eventually consistent, distributed system, simple CRUD (Create, Read, Update, Delete) operations have evolved into more sophisticated patterns to handle concurrent updates and maintain system integrity.

Understanding API Semantics

When interacting with the `kube-apiserver`, you have three primary methods for modifying existing resources. Choosing the right one is essential for building robust controllers.

1. Update (The "Put" Equivalent)

An **Update** operation is a full-object replacement. * **Mechanism:** The client sends the entire object definition to the API server. * **Concurrency:** It requires the `metadata.resourceVersion` to match the current version in `etcd`. If the version has changed (due to another controller acting on the object), the API server returns a `409 Conflict` error. * **Use Case:** Best used when you have the complete state and want to ensure you are modifying the specific version you read.

2. Patch (Partial Updates)

A **Patch** modifies only specific fields without requiring the full object body. Kubernetes supports several types of patching:

- **Strategic Merge Patch:** The default for built-in Kubernetes resources. It understands the schema of resources (e.g., how to merge items in a list).
- **JSON Patch (RFC 6902):** A sequence of operations (add, remove, replace, move, test).
- **Merge Patch (RFC 7386):** A simple JSON merge where `null` values explicitly delete fields.



When using Strategic Merge Patch, you can use the `$retainKeys` directive to explicitly drop fields from a list or map, providing granular control over object state that standard merging might otherwise preserve.

3. Server-Side Apply (SSA)

Server-Side Apply is the modern standard for declarative configuration management. It shifts the logic of merging from the client to the `kube-apiserver`.

- **Ownership:** SSA tracks "Field Managers." If two controllers attempt to modify the same field, the API server detects a conflict.

- **Efficiency:** It eliminates the "get-modify-put" race condition cycle that plagues traditional clients.
- **Observability:** Each field maintains metadata about which manager last updated it, making debugging complex reconciliation loops much easier.

Comparison Summary

Feature	Update	Patch
Server-Side Apply	Payload Size	Full Object
Partial	Partial/Declarative	Conflict Handling
Optimistic Locking	Depends on Patch type	Field Manager tracking
Logic Location	Client	Client

Hands-on Activity: Observing Patch Semantics

In this exercise, you will observe how a **Patch** behaves compared to a full **Update**.

Prerequisites

- A running Kubernetes cluster (or **kind**).
- **kubectl** configured.

Step 1: Create a Resource

Create a simple ConfigMap to experiment with:

```
kubectl create configmap demo-config --from-literal=key1=value1
```

Step 2: Perform a Strategic Merge Patch

Add a second key to the ConfigMap without including **key1** in your request:

```
kubectl patch configmap demo-config -p '{"data": {"key2": "value2"}}'
```

Verify the result:

```
kubectl get configmap demo-config -o yaml
```

Observation: **key1** is preserved because the Patch operation only modified the specific keys provided.

Step 3: Trigger a Conflict (Simulated)

To understand why we prefer Patches/SSA over Updates in controllers: 1. Open two terminals. 2. In Terminal A, get the object: `kubectl get cm demo-config -o yaml > obj.yaml`. 3. In Terminal B, edit

`obj.yaml` and change a field, then run `kubectl apply -f obj.yaml`. 4. Now, modify `obj.yaml` in Terminal A and run `kubectl apply -f obj.yaml`. **Result: You will trigger a conflict error because the `resourceVersion` has advanced.**

Troubleshooting Guidance

- **Conflict Errors:** If your controller receives a `409 Conflict`, you should implement a retry strategy: re-fetch the object, re-apply your logic, and try the Update again.
- **observedGeneration:** Always ensure your controller checks `.status.conditions[].observedGeneration` against the object's `.metadata.generation` to ensure you are acting on the latest intent, not stale cached data.

Week 2.2 — Going distributed: locks, leases, ownership, eventual consistency

Week 2.2 — Going distributed: locks, leases, ownership, eventual consistency

This module focuses on the challenges of operating multiple controllers in a distributed environment. It emphasizes building resilient, eventually consistent state machines by applying foundational distributed system principles to prevent conflicts and ensure reliable orchestration.

Key concepts include:

- **Idempotency:** Designing state machines that ensure multiple executions lead to the same result without side effects.
- **Synchronization Mechanisms:** Utilizing locks and Leases to manage access to shared resources and coordinate activities.
- **Leader Election:** Implementing patterns to ensure that only one controller instance performs a specific task at a given time.
- **Controller Sharding:** Distributing workload across multiple controller instances to improve scalability, utilizing strategies like resource sharding to handle large-scale clusters.

Designing idempotent state machines

Designing Idempotent State Machines

In the Kubernetes control plane, the core philosophy is **level-triggered reconciliation** rather than edge-triggered events. Because the network is unreliable and components may restart, your controllers must be designed as **idempotent state machines**.

An operation is idempotent if performing it multiple times results in the same state as performing it once. In Kubernetes, this means your **Reconcile** function must be able to handle being called repeatedly—even if the previous execution failed halfway through—without causing side effects or duplicate actions.

The Core Principles

To design idempotent systems, follow these three pillars:

1. **Observe before Act:** Never assume the current state. Always query the API server (or cache) to determine if an action is actually necessary.
2. **Deterministic Outputs:** Ensure that given the same input, your logic produces the same outcome. Avoid randomness or non-deterministic time-based calculations inside the reconciliation loop.
3. **Atomic State Transitions:** Utilize Kubernetes primitives like **Status** conditions and

ResourceVersions to ensure you are only acting upon the latest version of the truth.

Anatomy of an Idempotent Reconcile Loop

When writing a controller, the structure of your logic should follow this pattern:

```
func (r *MyReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    // 1. Fetch the latest instance
    obj := &v1alpha1.MyCustomResource{}
    if err := r.Get(ctx, req.NamespacedName, obj); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    // 2. Determine "what needs to happen" (Diffing)
    // Avoid blindly creating resources. Check if they exist first.
    if !isStateDesired(obj) {
        // 3. Perform atomic update
        err := r.applyChanges(ctx, obj)
        if err != nil {
            return ctrl.Result{}, err // Requeue on failure
        }
    }

    // 4. Update Status (Idempotent update of observed state)
    return ctrl.Result{}, r.updateStatus(ctx, obj)
}
```

Handling Conflicts: Optimistic Concurrency

Kubernetes uses **Optimistic Concurrency Control (OCC)**. Every resource has a **metadata.resourceVersion**. If you attempt to update a resource that has changed since you last read it, the API server will reject your request with a **409 Conflict** error.

- **The Idempotent Response:** Do not treat a conflict as a fatal error. Instead, treat it as a trigger to re-read the latest state and reconcile again. The controller-runtime library handles this automatically when you use **client.Update** or **client.Patch**.

Hands-on Activity: Designing for Idempotency

In this exercise, you will modify a reconciliation loop to move from "imperative" logic (which causes side effects) to "idempotent" logic.

Scenario

You are writing a controller that ensures a ConfigMap exists for every **App** resource.

Imperative (Wrong):

```
// This fails if the ConfigMap already exists!
err := r.Create(ctx, myConfigMap)
if err != nil {
    return err
}
```

Idempotent (Correct):

```
// 1. Attempt to find the object
found := &corev1.ConfigMap{}
err := r.Get(ctx, types.NamespacedName{Name: name, Namespace: ns}, found)

if errors.IsNotFound(err) {
    // 2. Only create if it does not exist
    return r.Create(ctx, myConfigMap)
} else if err != nil {
    return err
}

// 3. If it exists, verify it matches desired state.
// If not, perform an Update or Patch.
if !reflect.DeepEqual(found.Data, desiredData) {
    found.Data = desiredData
    return r.Update(ctx, found)
}
```

Lab Task

1. Using `controller-runtime`, create a `Reconcile` function that manages a `Deployment`.
2. Introduce a check that compares the current `Deployment` replicas with the `Spec.Replicas` of your Custom Resource.
3. Use the `Patch` strategy instead of `Update` to ensure that only changed fields are sent to the API server.
4. Verify by running the controller and manually deleting the `Deployment` while the controller is running; observe how the controller automatically recreates the resource without errors.

Locks and Leases for synchronization

Locks and Leases for Synchronization

In a distributed system like Kubernetes, multiple instances of a controller often run in parallel to ensure high availability. However, this introduces the risk of conflicting actions—such as two controllers attempting to modify the same resource simultaneously or performing redundant heavy computations. To ensure system stability and avoid "split-brain" scenarios, we use **Locks and Leases**.

Understanding Distributed Synchronization

In Kubernetes, we rely on **eventual consistency**. Because we cannot guarantee instantaneous global state, we use primitives that allow processes to agree on who is currently "in charge" of a specific task.

The Role of the Lease API

A **Lease** is a Kubernetes object (part of the **coordination.k8s.io** API group) that represents a lock. It provides a mechanism for a process to claim ownership of a resource for a specified duration (the **leaseDurationSeconds**).

- **Heartbeating:** The owner of the lease must periodically update the **renewTime**.
- **Expiration:** If the owner crashes or network partitions occur, the lease expires. Other candidates can then acquire the lease.
- **Atomicity:** By using the Kubernetes API server as the source of truth, we leverage its optimistic concurrency control (via **resourceVersion**) to ensure that only one controller successfully claims the lease.

Leader Election Mechanism

Leader election is the most common use case for Leases. It ensures that out of N controller replicas, only one performs the "active" reconciliation, while others remain in standby mode.

```
import (
    "k8s.io/client-go/tools/leaderelection"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
)

// The controller defines a lock on a Lease object
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{Name: "my-controller-lock", Namespace: "kube-
system"},
    Client:    client,
    LockConfig: resourcelock.ResourceLockConfig{
        Identity: podName,
    },
}

// Start the leader election loop
leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
    Lock:          lock,
    ReleaseOnCancel: true,
    LeaseDuration: 15 * time.Second,
    RenewDeadline: 10 * time.Second,
    RetryPeriod:   2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) { /* Run controller logic */ },
        OnStoppedLeading: func() { /* Cleanup */ },
    },
})
```

```
}),  
})
```

Hands-on Activity: Observing Leader Election

In this activity, we will deploy a multi-replica deployment and observe how the Lease object manages the leader.

Prerequisites

- A running Kubernetes cluster.
- `kubectl` configured with cluster-admin access.

Step 1: Create the Lease

Create a manifest to initialize the lock.

```
apiVersion: coordination.k8s.io/v1  
kind: Lease  
metadata:  
  name: example-leader-lock  
  namespace: default
```

Step 2: Simulate Multiple Replicas

Deploy two instances of a sample application that uses the `client-go/tools/leaderelection` library to compete for the `example-leader-lock`.

1. Deploy the pods: `kubectl apply -f controller-deployment.yaml` (ensure `replicas: 2`).
2. Check which pod owns the lease:

```
kubectl get lease example-leader-lock -o yaml
```

3. Observe the `holderIdentity` field.

Step 3: Trigger a Failover

1. Identify the leader pod: `kubectl get pods`.
2. Delete the leader pod: `kubectl delete pod <leader-pod-name>`.
3. Immediately inspect the Lease object again:

```
kubectl get lease example-leader-lock -o yaml
```

4. **Observation:** You will see the `holderIdentity` update to the name of the remaining pod once the `leaseDuration` has passed and the new leader performs its first heartbeat.

Best Practices and Gotchas

- **Clock Skew:** Never rely on system time for logic; always use the Kubernetes API server's time (via the `Lease` object) to avoid issues with drift.
- **Fencing:** Distributed locking is not a silver bullet. If your controller performs external I/O (e.g., updating a database outside K8s), ensure that your external systems also respect the lease ownership to prevent "zombie" processes from corrupting state.
- **Resource Contention:** Avoid creating a unique lease for every single custom resource. Instead, group resources or shard your controllers to minimize API server load.

Leader election mechanisms

Leader Election Mechanisms in Kubernetes Controllers

In a distributed system, running multiple instances of a controller is often necessary for high availability (HA). However, if multiple controllers attempt to perform the same reconciliation logic simultaneously, they can cause race conditions, duplicate operations, or inconsistent state in the target resources.

To prevent this, Kubernetes uses **Leader Election**, a mechanism that ensures only one instance of a controller (the "leader") is actively managing the workload, while others remain in a standby state.

Distributed Locking via Leases

Kubernetes implements leader election using the `coordination.k8s.io/v1` API, specifically through `Lease` objects. A Lease is a lightweight resource that acts as a distributed lock.

How Leader Election Works

1. **Contention:** Multiple controller instances attempt to acquire or renew a Lease object.
2. **Atomic Updates:** Using optimistic locking, only one controller can successfully update the Lease object's `holderIdentity` and `leaseDurationSeconds` fields at a time.
3. **Heartbeating:** The leader periodically updates the `renewTime` in the Lease.
4. **Failsafe:** If the leader crashes or becomes network-partitioned, it stops updating the Lease. Once the `leaseDurationSeconds` expires, other waiting instances perceive the leader as "dead" and attempt to acquire the lock themselves.

Implementing Leader Election with `client-go`

The `k8s.io/client-go/tools/leaderelection` package provides the standard tooling to integrate leader election into your custom controllers.

Prerequisites

Ensure your controller has the necessary RBAC permissions to create and update Leases:

```
rules:
- apiGroups: ["coordination.k8s.io"]
  resources: ["leases"]
  verbs: ["get", "create", "update", "patch"]
```

Step-by-Step Implementation

To implement leader election, you must wrap your controller's main loop in a `leaderelection.LeaderElector` configuration.

1. **Initialize the Resource Lock:** Create a lock object (usually a Lease) that the instances will fight over.
2. **Define Callbacks:** Use the `LeaderCallbacks` struct to define what happens when an instance becomes the leader or loses leadership.

```
import (
    "k8s.io/client-go/tools/leaderelection"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
)

// Define the lock
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{Name: "my-controller-lock", Namespace: "kube-
system"},
    Client:    clientSet.CoordinationV1(),
    LockConfig: resourcelock.ResourceLockConfig{Identity: podName},
}

// Start the election
leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
    Lock:          lock,
    ReleaseOnCancel: true,
    LeaseDuration: 15 * time.Second,
    RenewDeadline: 10 * time.Second,
    RetryPeriod:   2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // Start your controller's Run loop here
            runController(ctx)
        },
        OnStoppedLeading: func() {
            log.Info("Leader lost")
        },
    },
},
```

```
} )
```

Key Design Considerations

- **Idempotency:** Even with leader election, your reconciliation logic must be idempotent. External factors (such as network jitter or race conditions during the handover) can occasionally result in two controllers "thinking" they are leaders for a brief moment.
- **Lease Duration:** Setting the `LeaseDuration` too short can lead to "flapping," where the leader is constantly changed due to minor network delays. Setting it too long increases the recovery time during an actual controller crash.
- **Eventual Consistency:** Leader election provides **mutual exclusion**, but it does not replace the requirement for eventual consistency. The controller should always verify the state of the world before applying changes.

Hands-on Activity: Observing Leader Election

1. **Deploy the Controller:** Deploy two replicas of your controller pod into the cluster.
2. **Verify Lock:** Use `kubectl get lease -n kube-system` to see the `my-controller-lock` resource.
3. **Check Holder:** Note the `holderIdentity` in the lease description.
4. **Simulate Failure:** Delete the pod currently holding the lease.
5. **Observe Handover:** Watch the logs of the second pod. It will automatically detect the loss of the leader, wait for the lease to expire, and promote itself to the new leader.

Controller sharding strategies

Controller Sharding Strategies

In large-scale Kubernetes environments, a single controller instance can become a bottleneck. As the number of managed resources grows into the thousands or tens of thousands, the processing latency of the reconciliation loop increases, and the memory footprint may exceed reasonable limits. **Controller sharding** is the architectural pattern used to partition the workload across multiple controller instances, ensuring horizontal scalability and high availability.

The Need for Sharding

When a controller manages a large set of objects (e.g., thousands of `Ingress` resources), several issues arise:

- **Reconciliation Latency:** The time taken to process the queue increases as more events flood the system.
- **API Server Pressure:** Every controller instance maintains its own set of informers. High-frequency listing and watching can overwhelm the `kube-apiserver`.
- **Blast Radius:** If a single controller instance crashes or enters a "crash loop," the entire system state for all managed resources becomes stale.

Sharding allows us to distribute the "responsibility" for specific subsets of resources, effectively partitioning the workload.

Architectural Approaches to Sharding

There are two primary strategies for sharding controllers in Kubernetes:

1. Static Sharding

In this model, the workload is partitioned based on a property of the object, such as a hash of the object's name or a label.

- **Mechanism:** Each controller instance is assigned a unique ID. When an event arrives, the controller calculates `hash(resource_name) % total_shards`. If the result matches its assigned ID, the controller processes the object; otherwise, it ignores it.
- **Pros:** Simple to implement; no coordination required between instances.
- **Cons:** Resharding (changing the number of instances) causes a re-calculation of all assignments, which can trigger massive reconciliation churn.

2. Dynamic Sharding (Lease-based)

Dynamic sharding uses the `Lease` API (from the `coordination.k8s.io` group) to coordinate ownership of shards.

- **Mechanism:** Controllers compete for "shard leases." Once a controller acquires a lease for a specific shard, it becomes the exclusive owner of that segment of the workload.
- **Pros:** Highly resilient; if a controller dies, another instance can acquire the expired lease, ensuring automatic failover.
- **Cons:** Requires managing the coordination layer (etcd/API server) and handling potential race conditions during lease handover.

Case Study: OpenShift Ingress Controller

The OpenShift `ingress-controller` provides a practical example of sharding. To manage high-traffic clusters, OpenShift allows operators to shard routes across multiple Ingress Controller deployments.

- **Label-based selection:** You can configure an Ingress Controller to only claim `Routes` that match a specific label (e.g., `ingress-shard: public`).
- **Availability:** By using overlapping shards, administrators can ensure that if one shard is overwhelmed or unreachable, traffic might still be routed or processed via secondary shards, depending on the load balancer configuration.

Hands-on Activity: Planning a Sharded Controller

In this exercise, you will design the sharding logic for a hypothetical "CustomConfig" controller.

1. **Define the Shard Key:** Determine the partitioning criteria. For our example, we will use a label on the resource: `sharding.example.com/id`.

2. **Implement Filter Logic:** In your controller's `EventHandler`, ensure you verify ownership before adding the item to the workqueue:

```
func (c *Controller) shouldProcess(obj interface{}) bool {
    metaObj := obj.(metav1.Object)
    shardID := metaObj.GetLabels()["sharding.example.com/id"]

    // Check if this shardID belongs to this instance
    return shardID == c.myShardID
}
```

3. **Reflect on Reconciliation:** Consider what happens if a resource is created **without** the label.
- **Challenge:** Should it be ignored? Assigned to a "default" shard?
 - **Best Practice:** Always define a fallback or "default" behavior for unassigned resources to avoid orphaned objects.

Summary for Distributed Controllers

- **Use Leader Election** (via `client-go/tools/leaderelection`) when you have a singleton controller to ensure high availability.
- **Use Sharding** when the volume of resources exceeds the processing capacity of a single controller.
- **Always aim for idempotency.** Because network partitions or timing issues might cause two controllers to briefly think they own a resource, your `Reconcile` function must handle concurrent updates gracefully using optimistic locking.



Further Reading

Refer to the [Kubernetes Controller Sharding](#) documentation for advanced implementations involving informer filtering and custom predicates.

Week 3.1 — Taking more control: Admissions and beyond the kube-apiserver to writing your own API Server

Week 3.1 — Taking more control: Admissions and beyond the kube-apiserver to writing your own API Server

This module focuses on advanced Kubernetes extensibility, moving beyond basic Custom Resource Definitions (CRDs) to implement deep integration with the cluster control plane.

- **Admission Control:** Implementing Validating and Mutating webhooks to intercept, inspect, and modify API requests before they are persisted.
- **Policy Enforcement:** Understanding the functional distinctions between native admission webhooks and dedicated policy engines.
- **API Aggregation Layer:** Utilizing the aggregation layer to register custom API servers, allowing them to serve their own API groups alongside the core Kubernetes APIs.
- **API Discovery:** Managing and exposing API metadata to ensure clients can discover and interact with custom resources effectively.
- **Custom API Server Implementation:** Best practices for building independent API servers that integrate seamlessly with the Kubernetes ecosystem, supported by practical case studies like the OLM package-manager.

Admission control webhooks (Validating and Mutating)

Admission Control Webhooks: Validating and Mutating

In the Kubernetes request lifecycle, the `kube-apiserver` acts as the gatekeeper. Before an object is persisted to `etcd`, it passes through a sequence of admission controllers. While many are built-in (compiled into the binary), **Admission Webhooks** allow you to extend this functionality dynamically by calling out to external HTTP services.

Understanding Admission Controllers

Admission controllers are plugins that intercept requests to the Kubernetes API server after authentication and authorization are successful, but before the object is stored.

There are two primary types of webhooks:

1. **Mutating Admission Webhooks:** These modify the request object before it is persisted. They are processed first. A classic example is injecting a "sidecar" container or adding default labels

to a Pod.

2. **Validating Admission Webhooks:** These validate the request object against defined policies. If the object violates a policy, the request is rejected with an error. They are processed after mutation, ensuring they see the final version of the object.



Order matters. Mutating webhooks are executed before validating ones to ensure that the validation logic acts upon the final, intended state of the object.

The Webhook Lifecycle

When a request (e.g., `kubectl apply`) hits the API server, the following flow occurs:

1. **Object Conversion:** The API server converts the object into an internal version.
2. **Mutation:** The `kube-apiserver` sends an `AdmissionReview` request to registered Mutating Webhooks. The webhooks return an `AdmissionResponse` containing a JSON Patch to modify the object.
3. **Object Schema Validation:** Built-in validation checks occur.
4. **Validation:** The `kube-apiserver` sends an `AdmissionReview` request to registered Validating Webhooks. The webhooks return an `AdmissionResponse` with `allowed: true` or `allowed: false`.
5. **Persistence:** If all checks pass, the object is saved to `etcd`.

Webhooks vs. Policy Engines

While you can write custom webhooks for specific logic, complex cluster governance is often better handled by dedicated Policy Engines like OPA Gatekeeper or Kyverno.

- **Custom Webhooks:** Best for highly specific, procedural logic that requires deep integration with your internal systems (e.g., calling a legacy database or a proprietary container registry service).
- **Policy Engines:** Best for declarative governance (e.g., "All Pods must have a cost-center label"). They provide a high-level DSL (Domain Specific Language) that is easier to audit and manage at scale compared to writing and maintaining custom Go binaries.

Hands-on Exercise: Implementing a Registry Validator

In this exercise, you will create a Validating Webhook that prevents users from deploying container images from unauthorized registries.

Prerequisites

- A running Kubernetes cluster (or `kind`).
- `cert-manager` installed (to handle the HTTPS certificates required by the API server).
- A Go development environment.

Step 1: Define the ValidatingWebhookConfiguration

This object tells the `kube-apiserver` which resources to watch and where to send the requests.

```

apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: registry-validator
webhooks:
- name: validate.registry.example.com
  rules:
    - apiGroups: ["apps"]
      apiVersions: ["v1"]
      operations: ["CREATE", "UPDATE"]
      resources: ["deployments"]
  clientConfig:
    service:
      name: registry-validator-svc
      namespace: default
      path: "/validate"
    admissionReviewVersions: ["v1"]
    sideEffects: None

```

Step 2: Implement the Webhook Server

Your binary must expose an HTTP endpoint that consumes `admission.k8s.io/v1 AdmissionReview` objects and returns a JSON response.

```

// Simplified logic for the /validate endpoint
func handleValidation(w http.ResponseWriter, r *http.Request) {
    // 1. Decode the AdmissionReview from the request body
    // 2. Extract the Deployment object from admissionReview.Request.Object
    // 3. Iterate through containers and check the image string
    // 4. If image does not start with approved registry prefix:
    //    return AdmissionResponse{Allowed: false, Result: {Message: "Unauthorized
Registry"}}
    // 5. Otherwise, return AdmissionResponse{Allowed: true}
}

```

Step 3: Verification

After deploying your server and the `ValidatingWebhookConfiguration`:

1. Attempt to create a Deployment using an image from `docker.io` (if unauthorized).
2. Observe the API server rejecting the request: `Error from server: admission webhook "validate.registry.example.com" denied the request: Unauthorized Registry`



Always ensure your webhook server is highly available. If the webhook is unreachable, the API server will either "Fail Open" or "Fail Closed" based on the `failurePolicy` field in your configuration. For security, `failurePolicy: Fail` is recommended.

Differences between webhooks and Policy Engines

Admission Webhooks vs. Policy Engines: Architectural Differences

In the Kubernetes ecosystem, ensuring that resources conform to security, compliance, and organizational standards is critical. While both **Admission Webhooks** and **Policy Engines** share the goal of governing what enters the cluster, they operate with different philosophies, complexities, and operational overheads.

Admission Webhooks: The Native Mechanism

Admission Webhooks are the low-level, native mechanisms provided by the Kubernetes API server to intercept requests. They are effectively HTTP callbacks that the `kube-apiserver` invokes during the request lifecycle.

There are two primary types of native webhooks:

- **Mutating Webhooks:** These allow you to modify the content of a resource object before it is persisted to `etcd`. Use cases include injecting sidecars, adding default labels, or configuring security contexts automatically.
- **Validating Webhooks:** These allow you to verify the content of a resource. If the object does not meet your criteria, the request is rejected with an error message, preventing the object from ever being stored.

Characteristics of Custom Webhooks

- **Implementation:** You must write, containerize, and deploy a custom HTTP server.
- **Language:** Can be written in any language (Go, Python, etc.) that can serve HTTPS endpoints.
- **Control:** Offers granular, programmatic control; you have full access to the request/response logic.
- **Complexity:** Requires manual management of certificates (TLS), high availability, and performance tuning, as the API server depends on your webhook's availability.

Policy Engines: The Abstraction Layer

Policy Engines (such as OPA/Gatekeeper or Kyverno) are specialized frameworks built **on top** of the native Admission Webhook mechanism. Instead of writing custom code for every policy, these engines provide a declarative framework to define rules.

Why use a Policy Engine?

Policy engines abstract away the "plumbing" of webhooks. They manage the registration, TLS certificates, and the logic of the webhook server for you.

- **Declarative Rules:** Instead of writing Go code, you define policies using domain-specific languages (e.g., Rego for OPA) or standard YAML manifests (Kyverno).
- **Built-in Auditing:** Most policy engines provide built-in auditing capabilities, allowing you to view which resources in your cluster currently violate specific policies without actively

blocking them.

- **"Dry Run" Capabilities:** Policy engines often allow for **Audit** mode, where non-compliant resources are logged but not rejected, enabling "shift-left" testing of security rules.
- **Centralized Governance:** They allow for easier management of complex, multi-rule governance policies compared to maintaining dozens of disparate custom webhook servers.

Comparison Summary

Feature Custom Admission Webhook Policy Engine :--- :--- :--- Logic Definition
Imperative (Code-based) Declarative (DSL or YAML) Operational Overhead High (HTTPS, HA, Certs) Low (Engine handles infrastructure) Performance High (Highly optimized code) Moderate (DSL interpretation overhead) Maintenance Developers must maintain code Policy authors maintain rules Best For Complex, bespoke integrations Compliance, security, and standards

When to choose which?

- **Use a Custom Admission Webhook** when you have a unique requirement that cannot be easily expressed via declarative rules, such as integrating with an external legacy system or performing complex stateful lookups that would be inefficient in a DSL.
- **Use a Policy Engine** for standard governance, security best practices (e.g., "all images must come from internal registry"), and compliance reporting. It significantly reduces the maintenance surface area and provides a consistent interface for cluster administrators.



Regardless of the choice, remember that the **kube-apiserver** waits for these responses. If your webhook or policy engine is slow or unreachable, it directly impacts the ability to create or update resources in your cluster. Always ensure your admission solution is highly available and performant.

API discovery and Aggregation Layer

API Discovery and the API Aggregation Layer

In the Kubernetes ecosystem, the **kube-apiserver** acts as the single point of entry. However, as the cluster grows in complexity, there is often a need to extend the API surface beyond native resources and Custom Resource Definitions (CRDs). This is achieved through **API Discovery** and the **API Aggregation Layer**.

API Discovery

The Discovery API allows clients (like **kubectl** or custom controllers) to understand what resources are available in the cluster, what operations are supported, and which API groups are exposed.

How it Works

When you run **kubectl api-resources**, your client is interacting with the Discovery endpoints.

Historically, this was a large, monolithic response. Modern Kubernetes has evolved toward **Aggregated Discovery**, which provides more efficient, scalable discovery information, especially in clusters with many CRDs or aggregated APIs.

- **Standard Discovery:** Accessed via `/api` and `/apis`.
- **Aggregated Discovery:** Uses the `apidiscovery.k8s.io` group to serve discovery information in a more structured, granular manner, reducing the payload size for large clusters.



To inspect aggregated discovery responses using `kubectl proxy`, you can fetch the discovery list with the appropriate headers:

```
oc proxy &
curl -v http://127.0.0.1:8001/api \
-H "Accept:
application/json;v=v2;g=apidiscovery.k8s.io;as=APIGroupDiscoveryList"
```

The API Aggregation Layer

The API Aggregation Layer allows you to extend the Kubernetes API by running your own specialized API server alongside the primary `kube-apiserver`. This is essentially "hosting" your own API inside the Kubernetes ecosystem.

Why use the Aggregation Layer?

While CRDs are excellent for managing simple stateful resources, they have limitations regarding complex validation, custom storage backends, or existing API implementations. The Aggregation Layer provides:

1. **Custom Storage:** Unlike CRDs (which always store in `etcd`), an aggregated API server can store data in any backend (e.g., a SQL database or a third-party service).
2. **Advanced Logic:** It allows for custom endpoints that don't fit the standard `GET/POST/PUT/DELETE` paradigm of Kubernetes resources.
3. **API Versioning:** Full control over version conversion and complex API machinery.

How it works

The `kube-apiserver` acts as a proxy. When a request hits the master API server: 1. The `kube-apiserver` checks its internal routing table. 2. If the requested path belongs to an `APIService` object (which defines the custom API), the request is forwarded to your specific API server. 3. Your server handles the logic and returns the response back to the user through the `kube-apiserver`.

Hands-on Activity: Exploring APIServices

In this activity, we will list the current registered API services in your cluster to see how the aggregation layer is currently utilized.

Prerequisites

- An active Kubernetes or OpenShift cluster.
- `kubectl` or `oc` CLI tool configured.

Steps

1. **List existing APIServices:** The aggregation layer registers custom APIs via `APIService` objects. View them using: `[source,bash] --- kubectl get apiservices ---`
2. **Inspect a specific service:** Look at the output of a core-managed service, such as `v1beta1.metrics.k8s.io`: `[source,bash] --- kubectl get apiservice v1beta1.metrics.k8s.io -o yaml ---` Note the `spec.service` field. This points to the namespace and name of the internal Service that acts as the "backend" for this API.
3. **Understand the Routing:** Observe the `status` section. The `Available` condition indicates whether the `kube-apiserver` can successfully communicate with your custom backend. If this status is `False`, the API aggregation layer will fail to route requests to that extension.

Further Learning

To implement your own API server using the aggregation layer, refer to the [kubernetes/sample-apiserver](<https://github.com/kubernetes/sample-apiserver>) repository. It provides the standard framework for building a server that satisfies the `k8s.io/apiserver` library requirements.

Implementing custom API servers

Implementing Custom API Servers

When the standard Custom Resource Definition (CRD) mechanism is insufficient for your needs, the Kubernetes API Aggregation Layer provides the extensibility required to introduce your own, fully functional API server into a Kubernetes cluster.

Why Build a Custom API Server?

While CRDs are the standard for most extensions, you might need a dedicated API server when:

- * **Complex Data Models:** Your API requires complex logic, non-standard storage, or non-Kubernetes-native structures that do not map well to the CRD schema.
- * **Custom Storage:** You need to store data in a system other than `etcd` (e.g., a SQL database or an external cloud service).
- * **Advanced API Semantics:** You require custom proxying, specialized conversion logic, or complex resource relationships that the standard CRD lifecycle cannot support.
- * **Independent Scalability:** You want to scale your API's control plane independently of the primary Kubernetes API server.

The API Aggregation Layer

The API Aggregation Layer allows you to "plug in" your own API server alongside the core Kubernetes API server. It uses an `APIService` object to register your server, which acts as a proxy:

1. A client sends a request to the main `kube-apiserver`.

2. The `kube-apiserver` recognizes that the URL path belongs to your registered `APIService`.
3. The `kube-apiserver` proxies the request to your custom server.

Hands-on Activity: Exploring API Discovery

Before building, it is essential to understand how the system discovers these aggregated APIs. You can query the discovery endpoint to see how aggregated APIs are exposed.



Ensure you have access to a running cluster and have `oc` or `kubectl` configured.

Step 1: Accessing the Discovery Endpoint

Use the `oc proxy` to securely reach the API server and request the `APIGroupDiscoveryList` to see how the system registers and groups APIs.

```
# Start the proxy in the background or a separate terminal
oc proxy &

# Query the discovery endpoint
curl -v http://127.0.0.1:8001/api \
  -H "Accept: application/json;v=v2;g=apidiscovery.k8s.io;as=APIGroupDiscoveryList"
```

Step 2: Reviewing Implementation Patterns

To understand how to structure your server, examine existing implementations. The `sample-apiserver` provides the scaffolding for a compliant implementation using `k8s.io/apiserver`.

1. **Review the Sample:** Clone the repository to see the boilerplate code: <https://github.com/kubernetes/sample-apiserver>
2. **Examine Real-World Case Studies:** The OLM (Operator Lifecycle Manager) package manager API server is a production-grade example of a custom API server.
 - Explore the code here: <https://github.com/openshift/operator-framework-olm/tree/main/cmd/package-server-manager>

Best Practices for Custom API Servers

- **Reusability:** Utilize the `k8s.io/apiserver` library. It handles the heavy lifting of authentication, authorization, and audit logging, ensuring your custom server behaves like a native Kubernetes component.
- **Compatibility:** Always ensure your server supports the standard Kubernetes `REST` patterns. If you deviate from these, tools like `kubectl` or `oc` may not interact with your resources correctly.
- **Security:** Because your API server will be part of the trusted control plane, treat its development with the same rigor as the core Kubernetes components. Implement proper RBAC and transport security (TLS).
- **Discovery:** Implement the `APIGroupDiscovery` interface correctly to ensure your resources are visible to discovery clients and UI tools.

Week 3.2 — The other stuff

Week 3.2 — The other stuff

This module addresses the essential ecosystem components that complement the core Kubernetes control plane, focusing on traffic management, identity, and cluster-level security.

- **Advanced Networking:** Examines the evolution of traffic management from Ingress to the modern Gateway API, alongside an introduction to Service Mesh concepts.
- **Authentication and Authorization:** Explores AuthN/AuthZ mechanisms, including RBAC, Node Authorizers, and custom authorization webhooks.
- **Cluster Security and Hardening:** Covers security implementation strategies such as OpenShift Security Context Constraints (SCC), native Kubernetes `SecurityContext`, SELinux, and seccomp profiles to enforce granular workload isolation.

Advanced networking: Ingress and Gateway API

Advanced Networking: Ingress and Gateway API

In the Kubernetes ecosystem, managing external access to services is a critical architectural requirement. While early implementations relied on simple port-mapping and NodePorts, the evolution of traffic management has moved toward sophisticated, API-driven models.

From Ingress to Gateway API

The Ingress API

The `Ingress` resource has served as the standard for exposing HTTP and HTTPS routes from outside the cluster to services within the cluster for many years.

- **Function:** It acts as a collection of rules that allow inbound connections to reach the cluster services.
- **Limitation:** The Ingress specification is relatively narrow, focusing primarily on L7 (HTTP/S) traffic. This forced developers to rely heavily on vendor-specific annotations (e.g., `nginx.ingress.kubernetes.io/rewrite-target`) to achieve advanced routing, resulting in fragmented configurations that were difficult to port between different Ingress Controllers.

The Gateway API

The **Gateway API** is an evolution of Kubernetes networking designed to address the shortcomings of the Ingress API. It is an open-source project managed by the Kubernetes Networking Special Interest Group (SIG-Network).

- **Role-Oriented Design:** Unlike Ingress, which is a single, monolithic resource, the Gateway API uses a set of hierarchical resources:
- **GatewayClass:** Defines a type of load balancer (e.g., a specific cloud provider's LB or an NGINX

implementation).

- **Gateway:** Represents an instance of a load balancer, requesting specific ports and protocols.
- **HTTPRoute/GRPCRoute:** Defines the actual routing rules (matching criteria and backends) that attach to a **Gateway**.
- **Expressiveness:** It supports advanced features natively, such as weighted traffic splitting, header-based routing, and cross-namespace references, without requiring custom annotations.

Hands-on Activity: Deploying a Gateway API Route

This exercise demonstrates how to route traffic to a service using the Gateway API rather than the legacy Ingress resource.

Prerequisites

- A Kubernetes cluster (e.g., OpenShift or Minikube).
- The Gateway API CRDs installed on your cluster.
- A Gateway-compatible controller (e.g., Gateway API for Istio or an Ingress controller supporting the API).

Step 1: Define a Gateway

The **Gateway** resource creates the entry point into your cluster.

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: external-gateway
  namespace: default
spec:
  gatewayClassName: istio # Or your chosen provider
  listeners:
  - name: http
    protocol: HTTP
    port: 80
```

Step 2: Define the HTTPRoute

Instead of embedding rules in an Ingress, we create an **HTTPRoute** that links the **Gateway** to a specific **Service**.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app-route
spec:
  parentRefs:
  - name: external-gateway
```

```
rules:
- matches:
  - path:
      type: PathPrefix
      value: /api
  backendRefs:
  - name: my-service
    port: 8080
```

Key Takeaways for Developers

- **Portability:** By using the Gateway API, your routing configurations are standardized, reducing dependency on vendor-specific "magic" annotations.
- **Decoupling:** Gateway API separates the **Infrastructure** (Gateway) from the **Application** (HTTPRoute), allowing platform teams to manage load balancers while developers manage their own application routing.
- **Extensibility:** The API is designed to support not just HTTP, but also TCP, UDP, and gRPC traffic, providing a unified interface for cluster networking.



For deep dives into advanced traffic management, refer to the [official Gateway API documentation](#). When implementing service-to-service communication within the cluster, consider exploring the [GAMMA initiative](#), which bridges the gap between traditional Gateway traffic management and Service Mesh capabilities.

Service Mesh concepts

Service Mesh Concepts

In a distributed microservices architecture, the network is not reliable, and managing service-to-service communication becomes a significant operational burden. As the number of services grows, challenges like observability, security (mTLS), and traffic control become difficult to implement within application code. This is where a **Service Mesh** provides a dedicated infrastructure layer.

The Service Mesh Architecture

A service mesh decouples network logic from the application logic. It typically operates using a "sidecar" proxy pattern, where a lightweight proxy (such as Envoy) is deployed alongside every service instance.

- **Data Plane:** Consists of the sidecar proxies that intercept all inbound and outbound network traffic for your services. They handle service discovery, health checking, routing, load balancing, authentication, and observability.
- **Control Plane:** The centralized management component that configures the data plane proxies. It manages traffic policies, security configurations, and certificate distribution for mTLS.

Core Capabilities

A service mesh provides capabilities that would otherwise need to be manually implemented in every language and framework across your polyglot environment:

1. **Traffic Management:** Fine-grained control over traffic, including canary releases, A/B testing, and traffic shifting (e.g., sending 5% of traffic to a new version).
2. **Security (mTLS):** Automatic mutual TLS (mTLS) for all service-to-service communication, ensuring encryption in transit and strong identity-based authentication without changing application code.
3. **Observability:** Uniform telemetry collection across the entire cluster. You gain immediate access to golden signals: latency, traffic, errors, and saturation.
4. **Resiliency:** Advanced failure recovery, including retries, timeouts, circuit breaking, and rate limiting.

Evolving toward Gateway API for Mesh (GAMMA)

Traditionally, service meshes and Ingress controllers were managed via different APIs. The **GAMMA (Gateway API for Mesh Management and Administration)** initiative is a sub-project of the Kubernetes Gateway API that aims to provide a unified standard for managing service mesh traffic.

Instead of managing separate CRDs for Ingress and Service Mesh, GAMMA aims to use the existing Gateway API resources ([Gateway](#), [HTTPRoute](#)) to configure mesh traffic, offering a consistent developer experience across cluster edge and internal service-to-service communication.

Hands-on Activity: Observing Mesh Traffic

In this activity, we will simulate the need for a service mesh by inspecting internal service communication.



Identify existing service traffic

This exercise requires an existing Kubernetes cluster with at least two services (e.g., [frontend](#) and [backend](#)).

1. Deploy a simple workload if needed:

```
kubectl create deployment backend --image=nginx
kubectl expose deployment backend --port=80
```

2. **Understand the current connectivity:** Without a service mesh, traffic flows directly via the Kubernetes Service ClusterIP. To see this, use [tcpdump](#) inside a debug container (requires [ephemeral-containers](#)):

```
kubectl debug -it <frontend-pod-name> --image=nicolaka/netshoot -- /bin/bash
# Inside the container, perform a curl to the backend
```

```
curl http://backend:80
```

3. **Discussion:** Note that in this baseline setup, there is no encryption, no retries, and minimal observability. Introducing a service mesh like Istio would automatically inject a sidecar (e.g., `istio-proxy`), which would:
- Intercept the `curl` request.
 - Upgrade the connection to mTLS.
 - Log the request metrics to your monitoring stack (e.g., Prometheus/Grafana) without the application knowing about the proxy.

Troubleshooting Tips

- **Sidecar Lifecycle:** If a service fails to reach another, check if the `istio-proxy` (or equivalent) container is running and ready.
- **Port Naming:** Many service meshes require specific port naming conventions in your `Service` definition (e.g., `name: http-web`) to correctly identify the protocol.
- **Namespace Injection:** Ensure the namespace is labeled for injection (e.g., `kubectl label namespace default istio-injection=enabled`).

AuthN/AuthZ in OpenShift

AuthN/AuthZ in OpenShift

In the context of the Kubernetes developer learning path, moving from vanilla Kubernetes to OpenShift introduces a more integrated, opinionated approach to Authentication (AuthN) and Authorization (AuthZ). While OpenShift is built on top of Kubernetes, it extends the core primitives to provide a robust, enterprise-grade identity management system.

Authentication (AuthN) in OpenShift

In vanilla Kubernetes, AuthN is often handled via static tokens, certificates, or OIDC providers. OpenShift introduces the **OAuth Server**, a built-in identity provider layer that simplifies the management of users and access tokens.

The OAuth Server

The OpenShift OAuth server manages authentication requests. Unlike Kubernetes, where you might have to manage `kubeconfig` files with embedded tokens, OpenShift users typically log in via a web console or `oc login`, which exchanges credentials for an OAuth access token.

- **Identity Providers (IDPs):** OpenShift allows you to integrate with external systems such as LDAP, GitHub, GitLab, Google, or OIDC.
- **User and Identity Objects:** OpenShift separates the concept of an `Identity` (the external user record) from the `User` (the internal representation of that user within the cluster).

Authorization (AuthZ) in OpenShift

OpenShift builds upon Kubernetes Role-Based Access Control (RBAC) but adds a critical layer of abstraction known as **Security Context Constraints (SCCs)**.

RBAC vs. SCCs

While RBAC governs **who** can perform actions on API objects (like `get`, `list`, `create`), SCCs govern **what** a Pod is allowed to do from a security perspective (e.g., can it run as root, can it access the host network, can it mount host volumes).

Security Context Constraints (SCCs)

SCCs define the set of conditions that a pod must run with to be accepted by the system. * **Pre-defined SCCs:** OpenShift comes with default SCCs (e.g., `privileged`, `anyuid`, `restricted-v2`). * **The SCC Admission Controller:** When a pod is submitted, the SCC admission controller evaluates the service account under which the pod is running and checks it against the available SCCs. If the pod violates the security constraints, the request is rejected.

Hands-on Activity: Inspecting SCCs and Roles

This activity demonstrates how to view the security permissions assigned to your service account.

1. List available Security Context Constraints:

```
oc get scc
```

2. Inspect a specific SCC to understand its constraints:

```
oc describe scc restricted-v2
```

3. Check which service accounts can use a specific SCC:

You will notice the `users` and `groups` fields. These fields define which entities are permitted to request this security context.

4. View RoleBindings for your current project:

```
oc get rolebindings
```

- **Observation:** Notice the difference between `RoleBinding` (namespace-scoped) and `ClusterRoleBinding` (cluster-scoped).

Key Concepts Summary

Concept	Description
OAuth Server	Centralized identity management for cluster access.

User/Identity	Mapping external auth providers to internal cluster objects.
RBAC	API-level access control (Verb + Resource).
SCC	Pod-level security constraints (Privileged mode, SELinux, UIDs).



When building controllers in OpenShift, ensure your **ServiceAccount** has the necessary **RoleBindings** and, if your pod requires specific privileges (like host access), ensure your project's **ServiceAccount** has been granted access to the appropriate SCC using `oc adm policy add-scc-to-user`.

Cluster security and hardening techniques

Cluster Security and Hardening Techniques

In a distributed system as complex as Kubernetes, security cannot be an afterthought. Hardening your cluster requires a defense-in-depth approach, ensuring that even if one layer is compromised, the blast radius remains contained.

This section covers the critical components of securing a Kubernetes environment, with a specific focus on the nuances found in OpenShift.

The Defense-in-Depth Model

Securing a cluster involves controlling three primary vectors: 1. **Identity and Access:** Who can access the API? (AuthN/AuthZ) 2. **Workload Isolation:** What can the container do? (SCCs, Seccomp, AppArmor) 3. **Cluster Perimeter:** How is traffic flowing into the cluster? (Network Policies, Ingress)

Identity and Access Control

Fine-Grained RBAC

Kubernetes Role-Based Access Control (RBAC) follows the principle of least privilege. Instead of broad `cluster-admin` access, use granular Roles and ClusterRoles.

- **Verbs:** Limit permissions to the specific actions required (`get`, `list`, `watch`) rather than `*`.
- **Resources:** Scope roles to specific namespaces to prevent cross-namespace privilege escalation.

AuthN/AuthZ in OpenShift

OpenShift extends Kubernetes security by integrating a centralized identity provider with the platform. * **Groups and Roles:** OpenShift uses native **Groups** objects that map to your corporate directory, allowing you to bind roles to entire teams rather than individual users. * **Cluster Roles:** OpenShift provides pre-defined cluster roles (e.g., `edit`, `view`, `admin`) that are fine-tuned for developer workflows.

Workload Hardening

Standard Kubernetes allows containers to run with significant privileges by default. Hardening involves restricting these capabilities.

Security Context Constraints (SCC)

OpenShift uses **Security Context Constraints (SCCs)** to control the permissions of pods. Unlike standard Kubernetes Pod Security Admissions (PSA), SCCs are highly granular.

- **Restricted SCC:** The default for most workloads. It prevents pods from running as root, prevents host path mounting, and limits Linux capabilities.
- **Privileged SCC:** Reserved for infrastructure components like CNI plugins or storage drivers.

Seccomp and Capabilities

- **Seccomp (Secure Computing Mode):** Filters the syscalls a process can make to the Linux kernel. A hardened profile denies dangerous calls like **reboot** or **mount**.
- **Linux Capabilities:** By default, Docker/CRI-O gives containers a subset of root capabilities. You should explicitly drop all capabilities and only add back the ones strictly required by your application.

Hands-on Activity: Enforcing Security Policies

In this activity, you will define a Pod that adheres to the principle of least privilege using a Security Context.

```
apiVersion: v1
kind: Pod
metadata:
  name: hardened-pod
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
  containers:
    - name: app
      image: nginx
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop:
            - ALL
        readOnlyRootFilesystem: true
```

Step-by-Step Instructions

1. **Verify the Context:** Deploy the manifest above to your cluster.
2. **Observe Constraints:** Attempt to execute a shell inside the pod (`oc exec -it hardened-pod -- /bin/bash`).
3. **Analyze Failure:** Observe how `readOnlyRootFilesystem: true` prevents the application from writing to the file system, thereby mitigating potential post-exploitation activity.
4. **SCC Check:** If you are on OpenShift, check which SCC your pod is using:

```
oc get pod hardened-pod -o jsonpath='{.metadata.annotations.openshift\.io/scc}'
```

Summary Checklist for Hardening

- ☐ **Network Policies:** Implement a "Default Deny All" policy for all namespaces.
- ☐ **API Security:** Ensure your `kube-apiserver` is not exposed to the public internet.
- ☐ **Image Security:** Use signed images and scan them for vulnerabilities during the CI/CD phase.
- ☐ **Audit Logging:** Enable Kubernetes audit logs to track who performed what action and when.
- ☐ **Secret Management:** Never store raw secrets in Git; utilize external Vault providers or OpenShift Secrets with encryption-at-rest.



Refer to **Kubernetes Security and Observability** by Creane and Gupta for deep dives into runtime security monitoring and eBPF-based defense mechanisms.

Week 4.1 — All about kube-scheduler

Week 4.1 — All about kube-scheduler

The **kube-scheduler** serves as the core component responsible for workload placement within a Kubernetes cluster. It evaluates node capacity and constraints to determine the optimal host for each pod.

Key concepts include:

- **Pod Scheduling & Lifecycle:** Managing the placement of pods, including de-scheduling tasks and integration with auto-scaling mechanisms.
- **Resource Management:** Enforcing container-level resource Requests and Limits to ensure efficient hardware utilization and cluster stability.
- **Dynamic Resource Allocation:** Providing advanced scheduling control for specialized hardware (e.g., GPUs) through dynamic resource slicing and allocation patterns.

Pod scheduling, de-scheduling, and auto-scaling

Week 4.1: Pod Scheduling, De-scheduling, and Auto-scaling

The **kube-scheduler** is the "matchmaker" of the Kubernetes control plane. It observes newly created Pods that have no assigned node and selects an optimal node for them to run on. This module explores how the scheduler makes these decisions, how pods are evicted or moved, and how Kubernetes automatically adjusts capacity.

The Scheduling Lifecycle

Scheduling is not a single action but a multi-stage pipeline. When a Pod is created:

1. **Filtering (Predicates):** The scheduler filters the list of all nodes to find those that meet the Pod's requirements (e.g., CPU/Memory requests, node selectors, taints/tolerations).
2. **Scoring (Priorities):** The scheduler ranks the filtered nodes based on a set of scoring functions (e.g., spreading pods across zones, balancing load).
3. **Binding:** The scheduler creates a **Binding** object in the API server to associate the Pod with the chosen node.

De-scheduling and Eviction

Sometimes, the initial placement of a Pod becomes suboptimal due to cluster changes. This is where **De-scheduling** and **Eviction** come into play.

- **Eviction:** An API-initiated action where the control plane signals a Pod to terminate (often used during node draining for maintenance).
- **The Descheduler:** Because the **kube-scheduler** only acts when a Pod is created, it does not move Pods once they are running. The **Kubernetes Descheduler** is an add-on that analyzes cluster

state and evicts Pods that violate policies (e.g., too many Pods on one node, or nodes that are significantly under/over-utilized).

Auto-scaling Mechanisms

Auto-scaling ensures that your infrastructure matches your demand. It operates at two levels:

- **Horizontal Pod Autoscaler (HPA):** Adjusts the **number of Pod replicas** based on observed CPU utilization or custom metrics.
- **Vertical Pod Autoscaler (VPA):** Adjusts the **resource requests and limits** of containers in a Pod to ensure they have exactly the right amount of resources.
- **Cluster Autoscaler (CA):** Adjusts the **number of nodes** in the cluster. When the scheduler cannot find a node for a pending Pod, the CA provisions a new node from the underlying infrastructure provider (AWS, GCP, Azure, etc.).

Hands-on Activity: Configuring Pod Affinity and HPA

In this exercise, you will observe how the scheduler respects placement constraints and how HPA reacts to load.

Step 1: Deploy a constrained Pod

Create a YAML file `constrained-pod.yaml` to ensure your Pod lands on a specific node type.

```
apiVersion: v1
kind: Pod
metadata:
  name: affinity-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values:
                  - us-east-1a
  containers:
    - name: nginx
      image: nginx
```

Step 2: Implement Horizontal Pod Autoscaler

Create an HPA that scales a deployment based on CPU usage.

```
kubectl autoscale deployment my-app --cpu-percent=50 --min=1 --max=10
```

Step 3: Verification

1. Verify the Pod placement: `kubectl get pod affinity-pod -o wide`.
2. Check the HPA status: `kubectl get hpa`.
3. **Troubleshooting Tip:** If a Pod remains in **Pending** state, use `kubectl describe pod <pod-name>` to view the **Events** section. The scheduler will explicitly state why a node was rejected (e.g., "0/3 nodes are available: 3 node(s) didn't match node selector").

Best Practices

- **Always set Requests and Limits:** The scheduler relies on these to make capacity decisions. If you don't set them, the scheduler cannot accurately predict resource contention.
- **Use Pod Disruption Budgets (PDBs):** If you are running critical workloads, define a PDB to ensure that during voluntary disruptions (like node upgrades), a minimum number of replicas remain available.
- **Avoid Over-committing:** While Kubernetes allows you to over-commit resources, be aware that the `kube-scheduler` doesn't know the actual real-time load; it only knows the requested values.

Managing resource Requests and Limits

Managing Resource Requests and Limits

In the Kubernetes ecosystem, the `kube-scheduler` is responsible for placing pods onto nodes that have sufficient resources to accommodate them. To perform this task effectively, Kubernetes relies on the definitions of **Requests** and **Limits** specified within your Pod manifests. Understanding these mechanisms is fundamental to achieving cluster stability and efficient bin-packing.

Theoretical Overview

Resources in Kubernetes are primarily managed based on CPU (measured in `millicores`) and Memory (measured in `bytes`).

Requests: The Guaranteed Floor

A **Request** is the amount of resource that a container is guaranteed to have. When the scheduler decides which node a pod will land on, it sums up the `requests` of all containers in the pod and ensures the node has that much capacity available. * If the node does not have enough unallocated requested resources, the pod will remain in a **Pending** state.

Limits: The Hard Ceiling

A **Limit** defines the maximum amount of a resource a container is allowed to consume. * **CPU:** If a container exceeds its CPU limit, it is throttled, but it will not be terminated. * **Memory:** If a container attempts to exceed its memory limit, it may trigger an Out-Of-Memory (OOM) kill, causing the container to restart.



Always define both `requests` and `limits`. If a container specifies a limit but no request, Kubernetes automatically sets the request equal to the limit. If neither is

specified, the pod is assigned a **BestEffort** Quality of Service (QoS), making it the first candidate for eviction during node resource pressure.

Hands-On Activity: Configuring Resource Constraints

In this activity, you will define resource constraints for a simple NGINX deployment and observe how they affect the scheduling lifecycle.

Step 1: Create a Resource-Constrained Pod

Create a file named `resource-pod.yaml` with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
  - name: nginx-container
    image: nginx
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Step 2: Apply and Verify

Apply the configuration to your cluster:

```
kubectl apply -f resource-pod.yaml
```

Verify the resources assigned to the container by inspecting the pod description:

```
kubectl describe pod resource-demo | grep -A 2 Resources
```

Step 3: Observe Scheduling Behavior

1. **Under-provisioning:** Try to create a pod with an impossibly high request (e.g., `cpu: "100"`).
2. Run `kubectl get pods` and notice the pod status remains **Pending**.
3. Run `kubectl describe pod <pod-name>` to see the **FailedScheduling** event, which confirms the scheduler could not find a node with sufficient capacity.

Troubleshooting Tips

- **QoS Classes:** Kubernetes classifies pods into **Guaranteed**, **Burstable**, and **BestEffort** based on your requests/limits. Use `kubectl get pod <name> -o yaml` and look for `status.qosClass` to debug eviction issues.
- **CPU Throttling:** If your application performance is sluggish despite low CPU usage, check for throttling. You can monitor the `container_cpu_cfs_throttled_seconds_total` metric via Prometheus to see if your **limits** are set too low.
- **OOMKills:** If your pod crashes frequently, check the exit code:

```
kubectl get pod <pod-name> -o  
jsonpath='{.status.containerStatuses[0].state.terminated.reason}'
```

If it returns **OOMKilled**, you must increase your memory **limits**.

Dynamic Resource Allocation

Dynamic Resource Allocation (DRA)

In traditional Kubernetes scheduling, the **kube-scheduler** makes placement decisions based primarily on CPU and memory requests defined in a Pod's specification. However, as cluster workloads become increasingly specialized—relying on hardware accelerators like GPUs, FPGAs, or high-performance NICs—the static nature of resources becomes a bottleneck.

Dynamic Resource Allocation (DRA) is a Kubernetes API designed to extend the resource model, allowing third-party drivers to manage the lifecycle of complex, non-standard hardware resources.

Why do we need DRA?

Historically, Kubernetes managed specialized hardware using the **Device Plugin** framework. While effective for simple "pass-through" scenarios, Device Plugins suffer from several limitations:

- **Inflexible Allocation:** Device plugins operate at the node level. The scheduler has limited visibility into the specific state of the hardware.
- **Static Resource Requests:** It is difficult to model resources that can be partitioned or shared dynamically (e.g., slicing a GPU into smaller virtual instances).
- **No "Pre-allocation" Logic:** With standard plugins, the resource is often allocated only when the Pod is scheduled to the node, leaving little room for complex pre-provisioning requirements.

DRA shifts this by introducing a **claim-based** model, where the scheduler interacts with a Resource Driver to determine if and how a specific piece of hardware can be assigned to a Pod.

Core Concepts

DRA introduces three primary custom resources:

- **ResourceClass:** Defines the "policy" for how a resource should be allocated. It acts as the

interface between the cluster administrator's intent and the specific driver.

- **ResourceClaim**: A request for resources made by a user or an application. The claim acts as the binding between the Pod and the actual hardware.
- **ResourceClaimTemplate**: Used within a Pod template to allow for the automatic creation of **ResourceClaims** when a Pod is instantiated (similar to how a **PersistentVolumeClaim** works with a **PersistentVolume**).

Hands-on Activity: Configuring a Resource Claim

In this exercise, you will create a **ResourceClaim** to request a specialized resource managed by a DRA-compliant driver.



This activity assumes your cluster has a DRA-compatible driver installed (e.g., the official Kubernetes DRA dummy driver or a vendor-specific GPU driver).

Step 1: Create a Resource Class

Define the class that dictates which driver handles the request.

```
apiVersion: resource.k8s.io/v1alpha2
kind: ResourceClass
metadata:
  name: gpu-class
driverName: my-vendor.com/gpu-driver
```

Step 2: Define the Pod with a Resource Claim

Instead of hardcoding resource requests in the container spec, use a **ResourceClaimTemplate** to request resources dynamically.

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-workload
spec:
  containers:
    - name: worker
      image: alpine
      command: ["sleep", "infinity"]
      resourceClaims:
        - name: my-gpu
          source:
            resourceClaimTemplateName: gpu-template
  ---
apiVersion: resource.k8s.io/v1alpha2
kind: ResourceClaimTemplate
metadata:
  name: gpu-template
```

```
spec:
  spec:
    resourceClassName: gpu-class
```

Step 3: Verify Allocation

Once the Pod is created, the scheduler triggers the DRA controller to fulfill the claim. Check the status of your claim:

```
kubectl get resourceclaim
kubectl describe resourceclaim <claim-name>
```

The **status** field will show whether the resource has been allocated and which node it has been bound to.

Troubleshooting DRA

If your Pod remains in **Pending** state with DRA:

1. **Check Driver Logs:** Ensure your vendor-specific DRA driver is running in the **kube-system** namespace.
2. **Inspect ResourceClaim Events:** Run `kubectl describe resourceclaim <name>` to check for allocation failures.
3. **Validate ResourceClass:** Verify that the **driverName** in your **ResourceClass** matches the driver installed on your nodes.
4. **Scheduler Awareness:** Ensure your control plane is running a version of **kube-scheduler** that supports the **DynamicResourceAllocation** feature gate (enabled by default in modern Kubernetes versions).

Week 4.2 — Ubiquitous topics!

Week 4.2 — Ubiquitous topics!

This module explores diverse control plane architectures and platform form factors designed to support specific operational requirements, from enterprise cloud environments to resource-constrained edge devices.

- **Control Plane Architectures:** Examination of alternative Kubernetes control plane implementations and models.
- **HyperShift:** An architectural approach for running hosted OpenShift control planes (nested clusters) to improve scalability and lifecycle management.
- **kcp:** A Kubernetes-like control plane optimized for multi-cluster management and alternative use cases.
- **Edge Form Factors:** Implementation of enterprise-grade Kubernetes on small-scale hardware using solutions such as MicroShift, k0s, and microk8s.

Alternative control plane architectures

Alternative Control Plane Architectures

In standard Kubernetes deployments, the control plane—comprising the `kube-apiserver`, `etcd`, `kube-scheduler`, and `kube-controller-manager`—is typically colocated with the workload cluster or deployed as a dedicated management cluster. However, as Kubernetes expands into edge computing, massive multi-tenancy, and resource-constrained environments, standard architectures often face limitations.

Alternative control plane architectures decouple these components, providing specialized form factors for distinct use cases.

The Evolution of Control Plane Form Factors

The "one size fits all" approach to the Kubernetes control plane is evolving into a spectrum of architectures tailored to specific operational requirements:

- **Hosted/Nested Control Planes (HyperShift):** By running the control plane as a workload inside an existing OpenShift cluster, HyperShift isolates the control plane into a separate namespace. This provides rapid cluster provisioning and high density, as the control plane for a "guest" cluster is just a set of pods.
- **Logical Control Planes (kcp):** Designed as a "Kubernetes-like" system, `kcp` provides a control plane that is not tied to a single cluster. It acts as a universal API surface that can project resources into multiple underlying physical clusters, ideal for global scale and multi-cluster abstraction.
- **Single-Binary & Edge Optimized (MicroShift, k3s, k0s):** These architectures collapse the control plane components into a single, lightweight binary.

- **MicroShift:** An enterprise-ready, minimal-footprint distribution of OpenShift designed specifically for edge devices.
- **k3s/k0s/microk8s:** Optimized for low-resource environments (IoT, CI/CD pipelines) by stripping away legacy code and optimizing storage engines for small-scale deployments.

Comparative Architecture Overview

Architecture Type	Use Case	Primary Advantage
HyperShift	Massive multi-tenancy	Resource efficiency; fast provisioning.
kcp	Global orchestration	API-first design; multi-cluster management.
MicroShift/k3s	Edge / IoT	Low memory footprint; operational simplicity.

Hands-On Activity: Analyzing Component Density

In this activity, you will compare the footprint of a standard control plane against a lightweight alternative.

Prerequisites

- A running Kubernetes cluster (e.g., `kind` or `minikube`).
- `kubectl` installed and configured.

Step 1: Inspect Standard Control Plane Footprint

1. Check the resource footprint of your current control plane:

```
kubectl get pods -n kube-system -l component=kube-apiserver
kubectl top pods -n kube-system
```

2. Note the resource overhead required to maintain the etcd and API server processes.

Step 2: Simulate Minimal Architecture Concepts

Understand the difference by examining a single-binary distribution (like k3s): 1. Download a k3s-compatible environment or inspect the process tree of a minimal distribution:

+

```
# On a host running a minimal distribution, list processes
ps aux | grep -E "kube-apiserver|k3s"
```

1. Observe how `k3s` encapsulates the controller-manager, scheduler, and API server into a single process, whereas the standard cluster runs them as distinct, heavy-weight pods.

Troubleshooting Guidance

- **Observability:** When using nested control planes like HyperShift, remember that logs for your guest cluster control plane are stored as container logs within the management cluster. Use

`kubectl logs -n <guest-cluster-ns> <control-plane-pod>` to debug.

- **Consistency:** In distributed "logical" control planes (like `kcp`), latency between the control plane and the workload cluster can impact the reconciliation loop. Monitor the `Informers` and `SyncTarget` status to ensure eventual consistency is being maintained.

HyperShift and kcp

Advanced Control Plane Architectures: HyperShift and kcp

As Kubernetes matures, the requirement for diverse deployment environments—ranging from hyperscale cloud clusters to resource-constrained edge devices—has led to the evolution of the control plane itself. This section explores two prominent architectural patterns: **HyperShift** and **kcp**.

HyperShift: Hosted Control Planes

HyperShift (also known as Hosted Control Planes for OpenShift) is an architecture designed to decouple the control plane from the worker nodes. In standard Kubernetes, the control plane (API server, etcd, scheduler, controllers) runs on the same infrastructure as the worker nodes.

In the HyperShift model: * **Management Cluster:** Runs the control plane components as pods within a standard Kubernetes/OpenShift cluster. * **Guest Cluster:** Contains only the worker nodes (kubelet and kube-proxy).

Why HyperShift?

1. **Reduced Resource Overhead:** By centralizing control planes, you eliminate the need to provision dedicated control plane nodes for every single cluster, significantly reducing the "tax" of running many small clusters.
2. **Speed:** Because control planes are just sets of pods, spinning up a new "cluster" is significantly faster—often taking seconds rather than minutes.
3. **Isolation:** Control planes are isolated by namespace/pod security boundaries, allowing for dense multi-tenancy.

kcp: The Kubernetes-like Control Plane

While HyperShift is about managing standard Kubernetes control planes, **kcp** is a project that provides a "Kubernetes-like" API surface without being a full-blown Kubernetes cluster. It is designed for scenarios where you need the Kubernetes API (and its powerful controller ecosystem) but don't need the overhead of nodes, pods, or the kubelet.

Core Concepts of kcp

- **API-Centric:** kcp focuses solely on the API server, etcd, and the controller-runtime. It provides the "Kubernetes experience" (CRDs, controllers, workspaces, and virtual clusters) without managing physical infrastructure.
- **Logical Clusters (Workspaces):** kcp introduces the concept of workspaces, allowing for massive multi-tenancy where a single kcp instance can host thousands of virtual control planes.

- **Synchronization:** kcp uses **syncer** agents that watch resources in the kcp control plane and push those states to physical "target" clusters. This allows you to manage thousands of downstream clusters from a single, unified API endpoint.

Comparative Summary

Feature	HyperShift	kcp
Primary Goal	Efficient management of full K8s clusters	API-centric management of distributed resources
Architecture	Control plane as Pods	Specialized API server + Syncer agents
Use Case	Multi-cluster OpenShift/K8s management	Global fleet management and SaaS control planes
Workload Type	Full K8s Workloads (Pods)	Abstracted resources synchronized to targets

Hands-on Exploration: Understanding Control Plane Separation

In this activity, we will simulate the logic of a decoupled control plane by inspecting how a controller interacts with a target cluster via a remote kubeconfig.



This exercise assumes you have **kubectl** access to two clusters: a "Management" cluster (where your controller logic runs) and a "Target" cluster (the compute plane).

1. **Step 1: Setup the Controller Context** Create a controller that watches a Custom Resource (CR) in your Management cluster.

```
// Simplified controller logic to watch a CR in Management cluster
func (r *MyReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    // 1. Fetch CR from Management Cluster
    // 2. Identify the target cluster from the CR spec
    // 3. Connect to target using dynamic client
}
```

2. **Step 2: Implementing the Syncer Logic** Instead of the controller running **inside** the target cluster, implement a syncer pattern where your Management cluster controller pushes the desired state to the Target cluster.

```
# Ensure your controller has access to target kubeconfig
kubectl --kubeconfig=target-cluster.yaml apply -f manifest.yaml
```

3. **Step 3: Verification** Observe that the **resourceVersion** of the object in the Management cluster remains the source of truth, while the Target cluster reflects the applied state. Note how

`observedGeneration` in your status field confirms the reconciliation loop has successfully propagated the change to the remote edge/target.

Troubleshooting Tips

- **Latency:** When dealing with remote control planes (HyperShift), network latency between the Management and Guest control plane can affect reconciliation times. Always check `status.conditions` to see if a controller is stuck in a "Progressing" state.
- **Context switching:** When debugging, ensure your `KUBECONFIG` is correctly set. Use `kubectl config current-context` to confirm you are targeting the Management plane rather than accidentally running commands against the Guest/Target plane.

Microshift and k3s form factors

MicroShift and k3s Form Factors

In the evolution of Kubernetes, the "one-size-fits-all" model of a standard control plane often proves too resource-heavy for constrained environments. When moving from the data center to the edge—industrial IoT gateways, retail kiosks, or automotive systems—engineers must prioritize footprint, resource efficiency, and simplified lifecycle management.

This section explores two prominent lightweight Kubernetes distributions: **k3s** and **MicroShift**, which are designed specifically for these "small-footprint" scenarios.

k3s: The Lightweight Distribution

Developed by Rancher (now SUSE), k3s is a highly available, certified Kubernetes distribution designed for production workloads in unattended, resource-constrained, and remote locations.

Key Characteristics

- **Single Binary:** k3s packages all components (kube-apiserver, kube-scheduler, kube-controller-manager, kubelet, and kube-proxy) into a single, small binary (< 100MB).
- **SQLite by Default:** While it supports external databases like etcd, it defaults to SQLite for smaller setups, significantly reducing the memory overhead compared to a full etcd cluster.
- **Component Consolidation:** It removes legacy code, cloud-provider plugins, and unnecessary drivers found in upstream Kubernetes, focusing strictly on the requirements of edge compute.
- **Optimized for ARM:** k3s has first-class support for ARM64 and ARMv7, making it ideal for devices like the Raspberry Pi or specialized industrial ARM hardware.

MicroShift: OpenShift at the Edge

MicroShift is a foundational project that brings the operational consistency of OpenShift to small-scale environments. It is effectively a "stripped-down" version of OpenShift designed to run on single-node or limited-resource edge devices.

Key Characteristics

- **OpenShift Consistency:** MicroShift provides a subset of the OpenShift Container Platform (OCP) APIs. This allows developers to use the same security policies, image streams, and workload definitions they use in their data center OpenShift clusters.
- **Integrated Optimization:** Unlike standard Kubernetes, where you might bolt on monitoring or networking tools, MicroShift comes with pre-configured, optimized components that are tuned for the limited resource profile of edge hardware.
- **Life-cycle Alignment:** It focuses on being easy to update and manage, fitting into the "immutability" patterns required for remote edge nodes where manual troubleshooting is difficult.
- **Comparison to HyperShift:** While HyperShift (Hosted Control Planes) separates the control plane into a management cluster to gain density, MicroShift is designed for **standalone** local operation where connectivity to a central management hub may be intermittent.

Comparative Analysis

Feature	k3s	MicroShift	Primary Goal
Edge-optimized OpenShift API			Lightweight standard Kubernetes
Footprint			Extremely low (Single binary)
API Compliance			Low (Optimized OCP subset)
Best For			CNCF Certified
			OCP Sub-set (High compatibility)
			General purpose IoT/Small clusters
			Enterprise edge requiring OCP parity

Hands-on Activity: Observing Footprint

In this exercise, you will compare the memory footprint of a lightweight cluster against a standard distribution.

Step 1: Deploy k3s

On a Linux-based virtual machine or node, install k3s using the quick-start script:

```
curl -sfL https://get.k3s.io | sh -
```

Step 2: Analyze Process Memory

Inspect the memory usage of the Kubernetes components. Note the consolidation of the server process:

```
# View the consolidated server process
ps aux | grep k3s
# Check memory consumption of the core components
top -p $(pgrep -f "k3s server")
```

Step 3: Discussion

Reflect on the `ps` output. * **Question:** How many distinct processes do you see compared to a standard `kubeadm` install (which would show separate processes for `etcd`, `kube-apiserver`, etc.)? *

Observation: You will notice that k3s runs as a single process, reducing the memory pressure caused by overhead in inter-process communication and multiple Go runtimes.

Troubleshooting Tips

- **Resource Exhaustion:** If your edge device is failing to schedule pods, check `journalctl -u k3s` or `journalctl -u microshift` for OOM (Out-of-Memory) signals.
- **Cgroups:** Ensure the underlying OS has `cgroup v2` enabled, as both distributions rely heavily on modern kernel resource management to keep their footprint small.
- **Network:** When deploying at the edge, ensure that your firewall allows the specific ports required for the control plane, as these light distributions often strip out complex network policy implementations to save RAM.

Exercises

Exercises

The practical curriculum focuses on applying advanced Kubernetes concepts through hands-on development:

- **Controller Implementation:** Build a custom controller from the ground up using `client-go` to understand resource synchronization.
- **Distributed Systems Logic:** Implement optimistic locking and leader election mechanisms using the `Lease` API to ensure high availability.
- **Admission Control:** Develop and deploy a custom validating webhook to enforce container registry policies within the cluster.

Week 1: Build a controller using client-go from scratch

Week 1 Exercise: Building a Kubernetes Controller from Scratch

In this exercise, you will move beyond the `kubectl` CLI and interact directly with the Kubernetes API using `client-go`. By the end of this lab, you will have built a basic controller that "watches" for resources and logs their state, providing a foundational understanding of the `client-go` event propagation loop.

Objective

Create a Go application that utilizes `client-go` to:

1. Initialize a connection to the Kubernetes API.
2. Setup an **Informer** to watch for Pod creation events.
3. Use a **Workqueue** to process those events reliably.

Prerequisites

- Go 1.22+ installed.
- Access to a Kubernetes cluster (e.g., Minikube, Kind, or OpenShift Local).
- `kubeconfig` configured to point to your cluster.

Part 1: The client-go Architecture

Before coding, understand the core components we are implementing:

- **Informers:** These maintain a local cache of objects. Instead of polling the API server, they use a `watch` stream to keep the cache in sync.
- **Listers:** Provide a read-only interface to the local cache maintained by the Informer.
- **Workqueue:** A queue that decouples the event detection (the informer) from the processing logic (the reconciler). It helps handle retries and deduplication.

Part 2: Step-by-Step Implementation

Step 1: Initialize the Project

Create your project directory and initialize the Go module:

```
mkdir k8s-controller-lab
cd k8s-controller-lab
go mod init k8s-controller-lab
go get k8s.io/client-go@latest
go get k8s.io/apimachinery@latest
```

Step 2: Boilerplate and Configuration

Use `client-go` to connect to your cluster using the standard `kubeconfig` lookup.

```
// main.go
package main

import (
    "flag"
    "path/filepath"
    "k8s.io/client-go/tools/clientcmd"
    "k8s.io/client-go/util/homedir"
)

func main() {
    var kubeconfig *string
    if home := homedir.HomeDir(); home != "" {
        kubeconfig = flag.String("kubeconfig", filepath.Join(home, ".kube", "config"),
            "path to kubeconfig file")
    }
    flag.Parse()

    config, _ := clientcmd.BuildConfigFromFlags("", *kubeconfig)
    // ClientSet allows interaction with standard resources
    clientset, _ := kubernetes.NewForConfig(config)

    // ... logic continues to setup Informer
}
```

Step 3: Setting up the Informer

The Informer is the heartbeat of your controller. We attach event handlers (Add, Update, Delete) to it.

```
import (
    "k8s.io/client-go/informers"
    "k8s.io/client-go/tools/cache"
```

```

)

// Inside main()
factory := informers.NewSharedInformerFactory(clientset, 10*time.Minute)
podInformer := factory.Core().V1().Pods().Informer()

podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: func(obj interface{}) {
        fmt.Println("Pod added!")
    },
})

// Start the factory
stopCh := make(chan struct{})
defer close(stopCh)
factory.Start(stopCh)
cache.WaitForCacheSync(stopCh, podInformer.HasSynced)

```

Part 3: Adding Resilience

In a production controller, you never process items directly in the `EventHandler`. If your logic fails, the event is lost.

1. **Workqueue Implementation:** Modify the `AddFunc` to push the resource `Key` (namespace/name) into a `workqueue.RateLimitingInterface`.
2. **Reconcile Loop:** Create a worker function that loops indefinitely, reads from the queue, and processes the logic.
3. **Error Handling:** If your logic fails to process a Pod, call `queue.AddRateLimited(key)` to try again with exponential backoff.

Troubleshooting Tips

- **Cache Latency:** Remember that Informers are eventually consistent. If you create a Pod and immediately try to list it from the cache, it may not be there yet.
- **Syncing:** Always use `cache.WaitForCacheSync` at startup. If your controller logic starts before the cache is warmed up, you will miss resources.
- **Resource Leaks:** If your code stops, ensure you close the `stopCh` to prevent goroutine leaks in your local environment.

Exercise Challenge

Extend the logic: Instead of just printing "Pod added!", check the Pod's status. If the Pod does not have any labels, log a warning stating: **"Pod {Name} is missing required labels!"**

Week 2: Implement optimistic locking and leader election via Lease

Week 2: Implementing Distributed Coordination

When running multiple replicas of a controller for high availability, you face a significant challenge: "split-brain" scenarios. If two controllers attempt to reconcile the same resource simultaneously, they can cause race conditions, unnecessary API churn, or conflicting state changes.

To solve this, we use **Leader Election** based on Kubernetes **Lease** objects.

Understanding Distributed Synchronization

In Kubernetes, we achieve coordination through the **coordination.k8s.io** API group.

- **Lease:** A **Lease** object represents a lock held by a specific participant in a distributed system. It contains a **holderIdentity** and a **renewTime**.
- **Optimistic Locking:** Instead of locking a database row (which is risky in distributed systems), we use the **resourceVersion** field of the **Lease** object. If the version has changed since we last read it, the update fails, forcing the controller to retry.
- **Leader Election:** This is essentially a "race" to claim and maintain a **Lease**. The leader periodically updates the **renewTime**. If the leader crashes, the **renewTime** expires, allowing a follower to take over.

Hands-on Activity: Adding Leader Election to your Controller

In this exercise, we will extend your existing **client-go** controller to ensure only one instance is active at a time using the **client-go/tools/leaderelection** package.

Prerequisites

- A functional controller built with **client-go**.
- **leaderelection** package imported: **k8s.io/client-go/tools/leaderelection**
- **resourcelock** package imported: **k8s.io/client-go/tools/leaderelection/resourcelock**

Step 1: Define the Lock Identity

Each controller instance needs a unique identity (usually its Pod name).

```
import (
    "os"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
)

// Get unique ID for this instance
id, _ := os.Hostname()
```

Step 2: Configure the Lease Lock

We define a `LeaseLock` that tells Kubernetes which object to use to track the leader.

```
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{
        Name:      "my-controller-lock",
        Namespace: "default",
    },
    Client: clientset.CoordinationV1(),
    LockConfig: resourcelock.ResourceLockConfig{
        Identity: id,
    },
}
```

Step 3: Start the Leader Election Loop

The `leaderelection.RunOrDie` function handles the lifecycle of the lock. It provides callbacks to trigger your controller logic only when the lock is acquired.

```
leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
    Lock:          lock,
    ReleaseOnCancel: true,
    LeaseDuration: 15 * time.Second,
    RenewDeadline: 10 * time.Second,
    RetryPeriod:   2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // This is where you start your Controller's Run/Reconcile logic
            runMyController(ctx)
        },
        OnStoppedLeading: func() {
            log.Printf("Leader lost, shutting down.")
            os.Exit(0)
        },
        OnNewLeader: func(identity string) {
            log.Printf("New leader elected: %s", identity)
        },
    },
})
```

Troubleshooting and Best Practices

- **Lease Duration:** Do not set this too short (frequent network jitter will cause constant re-elections) or too long (failover will be slow if a node dies).
- **Eventual Consistency:** Remember that the controller state machine must be **idempotent**. If the leader crashes, the new leader will read the current state of your `Memcached` resources and reconcile them from scratch.

- **Debugging:** Use `kubectl get lease my-controller-lock -o yaml` to inspect the `holderIdentity` and verify which replica currently holds the lock.

Further Reading

- [Kubernetes Leases Documentation](#)
- [Martin Kleppmann: How to do distributed locking](#) (Important reading for understanding why distributed locks are rarely perfect).

Week 3: Develop a custom Validating Webhook for container registry enforcement

Week 3: Develop a Custom Validating Webhook for Container Registry Enforcement

In this exercise, you will extend your Kubernetes expertise by intercepting API requests to enforce governance. Specifically, you will build a **Validating Admission Webhook** that acts as a gatekeeper, preventing the creation or update of resources that reference unauthorized container registries.

Prerequisites

- A running Kubernetes cluster (e.g., `kind` or `minikube`).
- `cert-manager` installed in your cluster (for managing webhook TLS certificates).
- Go 1.20+ installed.

Understanding Admission Webhooks

Kubernetes Admission Controllers are plugins that govern and enforce policies on objects during create, update, or delete operations. * **Validating Webhooks:** Allow you to reject requests that do not meet your criteria. * **Mutating Webhooks:** Allow you to modify objects (e.g., adding default sidecars) before they are persisted to `etcd`.

Hands-on Lab: Registry Enforcement Webhook

Step 1: Define the Webhook Logic

Create a Go HTTP server that implements the `admission.k8s.io/v1 AdmissionReview` interface. Your logic needs to inspect the `AdmissionRequest` object.

```
func handleRegistryValidation(w http.ResponseWriter, r *http.Request) {
    // 1. Decode AdmissionReview from r.Body
    // 2. Check if object contains image field
    // 3. Compare image prefix against allowed registries
    // 4. If invalid, set response.Allowed = false and provide a message
    // 5. Encode AdmissionReview and write to w
}
```

Step 2: Implement Registry Enforcement

The binary must accept a list of approved registries via command-line flags. Use the `flag` package to capture these inputs during startup.

```
var allowedRegistries []string

func init() {
    flag.Func("allowed-registries", "Comma-separated list of approved registries",
func(s string) error {
    allowedRegistries = strings.Split(s, ",")
    return nil
    })
}

func isImageAllowed(image string) bool {
    for _, reg := range allowedRegistries {
        if strings.HasPrefix(image, reg) {
            return true
        }
    }
    return false
}
```

Step 3: Configure the ValidatingWebhookConfiguration

You must register your webhook with the `kube-apiserver` by creating a `ValidatingWebhookConfiguration` object. This tells Kubernetes where to send the `AdmissionReview` requests.

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: registry-validator
webhooks:
- name: validate.registry.example.com
  rules:
    - operations: ["CREATE", "UPDATE"]
      apiGroups: [""]
      apiVersions: ["v1"]
      resources: ["pods"]
  clientConfig:
    service:
      name: webhook-service
      namespace: default
      path: "/validate"
```



```
admissionReviewVersions: ["v1"]
sideEffects: None
```

Step 4: Deploy and Verify

1. **Build and Containerize:** Create a Dockerfile for your webhook binary and push it to a registry.
2. **Deploy:** Apply your **Deployment** and **Service** to the cluster.
3. **Test:** Attempt to create a Pod using an unauthorized image:

```
kubectl run unauthorized --image=docker.io/library/nginx:latest
# Expected Result: Error from server: admission webhook
"validate.registry.example.com" denied the request
```

Key Considerations

- **Fail Policy:** In production, set **failurePolicy: Fail** (to block if the webhook is down) or **failurePolicy: Ignore** (to allow requests if the webhook is unreachable).
- **TLS Requirements:** The **kube-apiserver** requires HTTPS to communicate with your webhook. Ensure your **ValidatingWebhookConfiguration** includes the **caBundle** generated by **cert-manager**.
- **Testing:** Always use **dry-run** during development to verify your webhook behavior without accidentally locking yourself out of the cluster.

Recommended Reading

Recommended Reading

To deepen your understanding of Kubernetes development and distributed systems, the following resources are recommended:

- **Core Concepts:**
- **Programming Kubernetes** by Michael Hausenblas and Stefan Schimanski.
- **Kubernetes Operators** by Jason Dobies and Joshua Wood.
- **CNCF Operator Whitepaper**.
- **Security and Observability:**
- **Kubernetes Security and Observability** by Brendan Creane and Amit Gupta.
- **Distributed Systems Design:**
- **Designing Distributed Systems** by Brendan Burns, focusing on patterns and paradigms for scalable, reliable systems.

Programming Kubernetes (Hausenblas, Schimanski)

Book Spotlight: Programming Kubernetes (Hausenblas, Schimanski)

*To master the Kubernetes control plane and effectively extend the platform, you must move beyond using `kubectl` and begin interacting with the cluster as a distributed systems developer. This section introduces **Programming Kubernetes: Developing Cloud-Native Applications** by Michael Hausenblas and Stefan Schimanski—a foundational text for anyone looking to transition from a Kubernetes operator to a Kubernetes developer.*

Why this book is essential for the Learning Path

While official documentation covers how to **use** Kubernetes, this book focuses on how to **program against** it. It aligns directly with our learning objectives concerning the **Kubernetes Resource Model**, **API-centric declarative management**, and the creation of **custom controllers**.

Core Concepts Covered

The authors provide deep technical insights into the mechanics that power the Kubernetes control plane:

- **The Kubernetes API Server:** Understanding the request/response lifecycle, API aggregation, and how the control plane handles resources.
- **The Controller Pattern:** A deep dive into the reconciliation loop—how the system continuously

monitors the difference between the current state and the desired state.

- **Extensibility via Custom Resources:** Practical guidance on leveraging `CustomResourceDefinitions` (CRDs) to define your own API objects.
- **The client-go Library:** Essential for any developer aiming to build controllers. The book demystifies informers, listers, and indexers, which are the building blocks of efficient, event-driven Kubernetes applications.

How to leverage this for your training

As you progress through our **Week 1 and Week 2** modules, use this book as a technical reference to bridge the gap between abstract concepts and implementation:

Module	Focus Area
Chapter Correlation	Week 1.1
Kubernetes Resource Model	Understanding API machinery and the storage of objects in <code>etcd</code> .
Week 1.2	client-go Architecture
Implementing informers and workqueues to watch for cluster events.	Week 2.1
Controller-Runtime	Building reconciled state machines using the patterns established in the book.

Key Takeaways for Developers

If you are just starting your journey into Kubernetes development, focus on these three pillars emphasized by Hausenblas and Schimanski:

1. **Declarative Intent:** Always design your controllers to be idempotent. The system should be able to recover from crashes by re-reading the state and moving toward the objective without side effects.
2. **Edge-Driven vs. Level-Driven:** Learn why Kubernetes favors level-driven triggers (focusing on the "what is the current state") over edge-driven triggers (focusing on "what changed").
3. **Observability:** Building a custom controller requires deep visibility into the `kube-apiserver`. The book provides excellent patterns for logging and metrics to ensure your custom controllers are debuggable in production environments.

Suggested Hands-on Study Path

To get the most out of this resource alongside our curriculum, attempt the following workflow:

- **Step 1:** Read the introductory chapters on the Kubernetes API.
- **Step 2:** Experiment with `client-go` using the book's examples to list pods in your cluster.
- **Step 3:** Use the book's guidance on **informers** to build a small utility that prints an event every

time a ConfigMap is modified in your namespace.



Treat the code examples in the book as "living" documentation. As you progress to **Week 2.1**, compare the raw `client-go` approaches taught in the early chapters of the book with the higher-level abstractions provided by the `controller-runtime` library. Understanding both is critical for debugging complex scenarios.

Kubernetes Operators (Dobies, Wood)

Understanding Kubernetes Operators: Automating the Container Platform

This module focuses on the principles of **Kubernetes Operators**, as popularized by Jason Dobies and Joshua Wood in their seminal work **Kubernetes Operators: Automating the Container Orchestration Platform**.

An Operator is not just a piece of software; it is a **method of packaging, deploying, and managing a Kubernetes application**. By using custom resources and controllers, an Operator enables human-level operational knowledge to be baked directly into the cluster.

The Operator Pattern

The core philosophy of an Operator is to extend the Kubernetes API to handle domain-specific knowledge. While standard controllers manage built-in resources (like Pods or Deployments), Operators manage custom resources (CRDs) that represent entire applications or complex services.

Why Operators?

Standard Kubernetes controllers are excellent at scaling and self-healing individual components. However, they lack the "application awareness" required for stateful workloads. Operators bridge this gap by:

- **Encoding Operational Logic:** Automating tasks like database backups, schema migrations, or complex cluster bootstrapping.
- **Encapsulating State:** Providing the logic to maintain a system in a healthy state despite external variables.
- **Declarative Management:** Allowing users to define **what** they want the application to look like, while the Operator figures out **how** to achieve it.

Reconcile: The Heart of the Operator

At the center of every Operator is the **Reconciliation Loop**. As established in our earlier study of the Kubernetes API, the Operator continuously observes the cluster, compares the "actual state" to the "desired state," and performs operations to align them.

```
// Simplified pseudo-code for a reconcile loop
func (r *MyReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result,
error) {
    // 1. Fetch the custom resource
```

```

    instance := &v1alpha1.MyKind{}
    r.Get(ctx, req.NamespacedName, instance)

    // 2. Perform business logic/checks
    // 3. Update the Status of the CR
    // 4. Return result and potential error for requeue
    return ctrl.Result{}, nil
}

```

Hands-on Activity: Scaffolding an Operator with Kubebuilder

In this activity, we will initiate a new project using `kubebuilder` to create the boilerplate for a custom operator.

Prerequisites

- Go (latest stable version)
- `controller-runtime` and `kubebuilder` tools installed.
- An active Kubernetes cluster (or `kind`).

Step 1: Initialize the Project

Initialize a new directory and scaffold the controller structure:

```

mkdir my-operator
cd my-operator
kubebuilder init --domain example.com --repo github.com/example/my-operator

```

Step 2: Create a Custom API

Define a new Custom Resource Definition (CRD) called `AppService` with a group `apps` and version `v1alpha1`:

```

kubebuilder create api --group apps --version v1alpha1 --kind AppService

```

Follow the prompts to create both the Resource and the Controller.

Step 3: Implement the Reconcile Logic

Open the generated file `internal/controller/appservice_controller.go` and implement the logic to inspect your custom resource. Observe the structure provided by `controller-runtime`, which implements the interface required to watch and reconcile your new `AppService` objects.

Step 4: Verify

1. Generate manifests: `make manifests`
2. Install CRDs: `make install`

3. Run locally: `make run`

Expert Insight: Designing Idempotent State Machines

When building your operator, keep these two rules from the Dobies/Wood guide in mind:

1. **Idempotency:** Your `Reconcile` function must be safe to call multiple times. If your code creates a Deployment, ensure it checks if the Deployment already exists before trying to create it again.
2. **Status vs. Spec:** Always maintain a clear separation. The **Spec** is what the user wants; the **Status** is what the controller reports about the real world. Never allow the controller to overwrite the user's Spec, and never allow the user to manually edit the Status.

Further Reading

For a deep dive into the practical application of these patterns, refer to the following: * **Kubernetes Operators** (Dobies, Wood), Chapters 3-5: Detailed breakdown of the Operator pattern and custom resource lifecycles. * **CNCF Operator Whitepaper:** Essential reading for understanding the maturity levels of Operators.

CNCF Operator Whitepaper

CNCF Operator Whitepaper

The [CNCF Operator Whitepaper](#) serves as the foundational architectural guide for understanding how to automate the lifecycle of software running on Kubernetes. As you progress through the Kubernetes Developer Learning Path, this document is essential for moving beyond basic resource management and into the realm of complex system automation.

What is an Operator?

At its core, an Operator is a Kubernetes controller that uses Custom Resource Definitions (CRDs) to manage a specific application or service. The Whitepaper defines an Operator by its ability to:

- **Codify Operational Knowledge:** Encode the "human-operator" expertise into software (e.g., how to perform backups, upgrades, or scaling).
- **Extend the API:** Utilize custom resources to provide a declarative interface for domain-specific applications.
- **Implement Reconcile Loops:** Constantly observe the current state of the application and take action to match the desired state defined in the custom resource.

The Operator Capability Model

One of the most significant contributions of the CNCF Operator Whitepaper is the **Operator Capability Level** framework. This model helps developers measure the maturity of their automation:

Level	Description
1. Basic Install	Automation of application installation and basic configuration.
2. Seamless Upgrade	Managing version upgrades, both minor and major.
3. Full Lifecycle	Handling deep operational tasks like backups, restores, and failover.
4. Deep Insights	Providing advanced monitoring, metrics, alerts, and log analysis.
5. Auto Pilot	Advanced features like auto-scaling, self-healing, and anomaly detection.

Key Principles for Controller Design

Drawing from the whitepaper and our learning objectives, developers should focus on these distributed system principles when building Operators:

- **Idempotency:** Your **Reconcile** function must be able to run multiple times with the same input without causing side effects.
- **Eventual Consistency:** Controllers do not guarantee immediate state changes. The system should always trend toward the **Spec** defined in the CRD.
- **Failure Tolerance:** Since the control plane can experience network partitions, your controller must handle stale caches and requeue events appropriately.

Hands-on Activity: Analyzing the Capability Level

To better understand these concepts, perform a comparative analysis of an existing Operator.

1. **Task:** Select a well-known operator, such as the **Prometheus Operator** or the **Etcd Operator**.
2. **Examine:** Review their documentation against the **CNCF Operator Whitepaper** capability table.
3. **Document:**
 - What CRDs does it define?
 - Does it provide metrics (Level 4)?
 - Does it handle automatic failover/healing (Level 5)?
4. **Discussion:** Map the features you found to the **client-go** components (Informer, Lister, and Workqueue) that would logically be required to implement that level of capability.

Troubleshooting Guidance

When reviewing the whitepaper in the context of your own development:

- **Avoid "Over-Engineering":** Don't attempt Level 5 automation on your first controller. Focus on robust, idempotent Level 1 logic first.
- **Watch for Blocking Reconciles:** Never block the **Reconcile** loop. If a process takes a long time, leverage **RequeueAfter** in your controller logic to let other events process.
- **Logging vs. Events:** Ensure your controller generates Kubernetes Events for state transitions, as these are the primary way users gain visibility into the controller's decision-making process.



For a deeper dive into the implementation of these concepts, refer to **Kubernetes Operators** by Dobies and Wood, which bridges the gap between the whitepaper’s architectural theory and practical implementation using `kubebuilder`.

Suggested Reading

Suggested Reading

To deepen your understanding of Kubernetes internals, distributed systems design, and cluster security, refer to the following curated resources:

- **Kubernetes Security and Observability** (Creane & Gupta, 2021): A practical guide for implementing robust security postures and effective monitoring strategies.
- **Designing Distributed Systems** (Burns, 2024): An exploration of reusable software patterns and paradigms for building scalable and reliable systems on Kubernetes.

Kubernetes Security and Observability (Creane, Gupta)

Kubernetes Security and Observability

In the context of the Kubernetes Developer Learning Path, understanding how to secure and monitor your custom resources and controllers is as critical as writing the code itself. As outlined by Creane and Gupta in **Kubernetes Security and Observability**, the transition from managing standard resources to building custom extensions requires a shift in how we perceive the "trusted" boundaries of the cluster.

The Security-Observability Intersection

When developing custom controllers or admission webhooks, you are effectively expanding the control plane's surface area. According to Creane and Gupta, security in Kubernetes is not just about perimeter defense; it is about **defense-in-depth** within the API lifecycle.

Key Security Principles for Developers

- **Least Privilege for Controllers:** A controller requires a **ServiceAccount** with RBAC permissions. Developers must follow the principle of least privilege, granting only the specific **verbs** (get, list, watch, create, update, patch) on the specific resources required by the operator.
- **Admission Control as a Security Gate:** As we will explore in Week 3.1, **Validating** and **Mutating** webhooks are your first line of defense. They ensure that custom resources meet security policies (e.g., preventing the use of privileged containers) before they ever reach **etcd**.
- **Securing the API Server:** Since the **kube-apiserver** acts as the single point of entry for all modifications, securing its communication via TLS and ensuring proper **Authentication** and **Authorization** modules are enabled is paramount.

Observability: The Feedback Loop

Observability is the "eyes" for your distributed system. For a developer building controllers, observability is defined by three pillars: 1. **Metrics:** Exposing controller health via Prometheus endpoints. Are your workqueues backing up? How long does the **Reconcile** loop take? 2. **Logging:**

Structured logs allow you to trace the lifecycle of a resource through the API request/response flow.

3. **Tracing:** Distributed tracing (e.g., Jaeger) is essential when a request triggers multiple webhooks or cross-controller interactions, helping you identify where an "event propagation" delay might be occurring.

Practical Activity: Securing a Custom Controller

In this activity, we will define a `ClusterRole` to ensure our controller adheres to the principle of least privilege.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: my-custom-controller-role
rules:
- apiGroups: ["my.domain.com"]
  resources: ["customresources"]
  verbs: ["get", "list", "watch", "update", "patch"]
- apiGroups: [""] # Core API group
  resources: ["events"] # Required for logging controller actions
  verbs: ["create", "patch"]
```

Step-by-Step Instructions

1. **Define the Scope:** Identify the exact resources your controller needs to reconcile. Avoid using wildcard (*) verbs.
2. **Audit the RBAC:** Before deploying, use `kubectl auth can-i` to verify the permissions of your controller's ServiceAccount:

```
kubectl auth can-i update customresources --as=system:serviceaccount:default:my-controller
```

3. **Implement Logging:** Ensure your controller logs the `UID` and `Generation` of the resource it is reconciling. This provides the audit trail required for observability.

Troubleshooting Guidance

- **Unauthorized Errors:** If your controller fails to reconcile, check the `kube-apiserver` audit logs. It will show the specific `ServiceAccount` attempting an action and the denied `verb` or `resource`.
- **Performance Bottlenecks:** If your controller logs suggest a backlog in the workqueue, check your `Informers`. Excessive `resync` periods can lead to unnecessary API pressure, which can be identified by monitoring the `controller_runtime_reconcile_time_seconds` metric.



For further study, refer to **Kubernetes Security and Observability** (Creane, Gupta) to understand how to map these concepts to the broader cluster infrastructure, particularly regarding OpenShift-specific hardening techniques

Designing Distributed Systems (Burns)

Designing Distributed Systems: A Kubernetes Developer's Perspective

Drawing from Brendan Burns' foundational work, **Designing Distributed Systems**, we explore the patterns required to manage the inherent complexity of Kubernetes-based environments. As a Kubernetes developer, understanding these patterns is the difference between building a fragile script and engineering a robust, self-healing system.

The Core Philosophy: Patterns over Components

In a distributed system like Kubernetes, you cannot rely on synchronous communication. Instead, you must design for **eventual consistency** and **decoupled execution**. Burns emphasizes that by combining primitive building blocks—such as the Sidecar, Ambassador, and Work Queue—we can achieve the same complexity as traditional monolithic systems, but with the added benefits of horizontal scalability and resilience.

Key Architectural Patterns

- **The Sidecar Pattern:** Attaching a secondary container to a primary application container to extend its functionality (e.g., logging, monitoring, or service mesh proxies) without modifying the application code.
- **The Ambassador Pattern:** A proxy container that hides the complexity of external service connections, allowing the application to communicate with a local endpoint.
- **The Adapter Pattern:** Standardizing the output of an application so that it can be consumed by monitoring or logging systems, regardless of the application's internal format.
- **The Work Queue System:** Utilizing `client-go` informers and work queues to manage tasks asynchronously, ensuring that your controller doesn't block while waiting for API server responses.

Practical Application: Implementing a Sidecar Pattern

In this exercise, we will implement a simple Sidecar pattern. We will use a primary Nginx container and a "Log-Router" sidecar that reads the Nginx access logs and forwards them to a standard output, simulating a logging agent.

Hands-on Activity: Deploying a Sidecar

1. **Define the Pod Specification** Create a file named `sidecar-pod.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-with-sidecar
spec:
```

```
containers:
- name: nginx
  image: nginx
  volumeMounts:
  - name: log-storage
    mountPath: /var/log/nginx
- name: log-router
  image: busybox
  command: ["/bin/sh"]
  args: ["-c", "tail -f /var/log/nginx/access.log"]
  volumeMounts:
  - name: log-storage
    mountPath: /var/log/nginx
volumes:
- name: log-storage
  emptyDir: {}
```

1. Deploy and Verify Apply the manifest and inspect the shared volume:

```
kubectl apply -f sidecar-pod.yaml
# Observe the sidecar reading logs from the shared volume
kubectl logs nginx-with-sidecar -c log-router
```

Designing for Failure: Locks and Leases

When applying these distributed patterns to your own custom controllers, you will encounter the **"Split Brain"** scenario. If you run multiple instances of a controller for high availability, they may attempt to reconcile the same resource simultaneously, leading to race conditions.

To prevent this, we move from simple loops to **Leader Election** using Kubernetes **Leases**.



Refer to the [coordination.k8s.io](https://kubernetes.io/docs/concepts/configuration/secret/) API group. By using the **Lease** object, your controllers can participate in a leader election process. Only the current leader reconciles the state, while others standby, ensuring that your distributed system maintains atomicity and consistency.

Further Exploration

To master these concepts, align your development strategy with these principles: * **Idempotency**: Every reconcile action must be safe to repeat, even if the previous attempt failed halfway. * **Observability**: If a system is distributed, it must expose metrics via the Adapter pattern to prove it is working as intended. * **Eventual Consistency**: Never assume the API server state matches the cluster state instantaneously; always rely on the Informer cache.

Appendix

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)